

+ 26

Back To Basics Code Analysis

ALEXSANDRO THOMAS



20
26



About me

Alexsandro Thomas

Senior Software Engineer

Languages nerd, GPGPU enthusiast

 alexsandrothomas

 northy

Slides URL: talks.alexsand.ro/b2b-code-analysis



Introduction: Code Analysis

A tiny program: `humidity-stats`

Loads integer humidity readings from a file, then prints:

A tiny program: `humidity-stats`

Loads integer humidity readings from a file, then prints:

 readings.txt

```
20 22 22 25 31 18 40 35 22 27
```

A tiny program: `humidity-stats`

Loads integer humidity readings from a file, then prints:

- minimum, maximum, and mean

 readings.txt

```
20 22 22 25 31 18 40 35 22 27
```

```
min 18 max 40 mean 26
```

A tiny program: `humidity-stats`

Loads integer humidity readings from a file, then prints:

- minimum, maximum, and mean
- the number of distinct readings

 readings.txt

```
20 22 22 25 31 18 40 35 22 27
```

```
min 18 max 40 mean 26  
distinct 8
```

A tiny program: humidity-stats

Loads integer humidity readings from a file, then prints:

- minimum, maximum, and mean
- the number of distinct readings
- a histogram

readings.txt

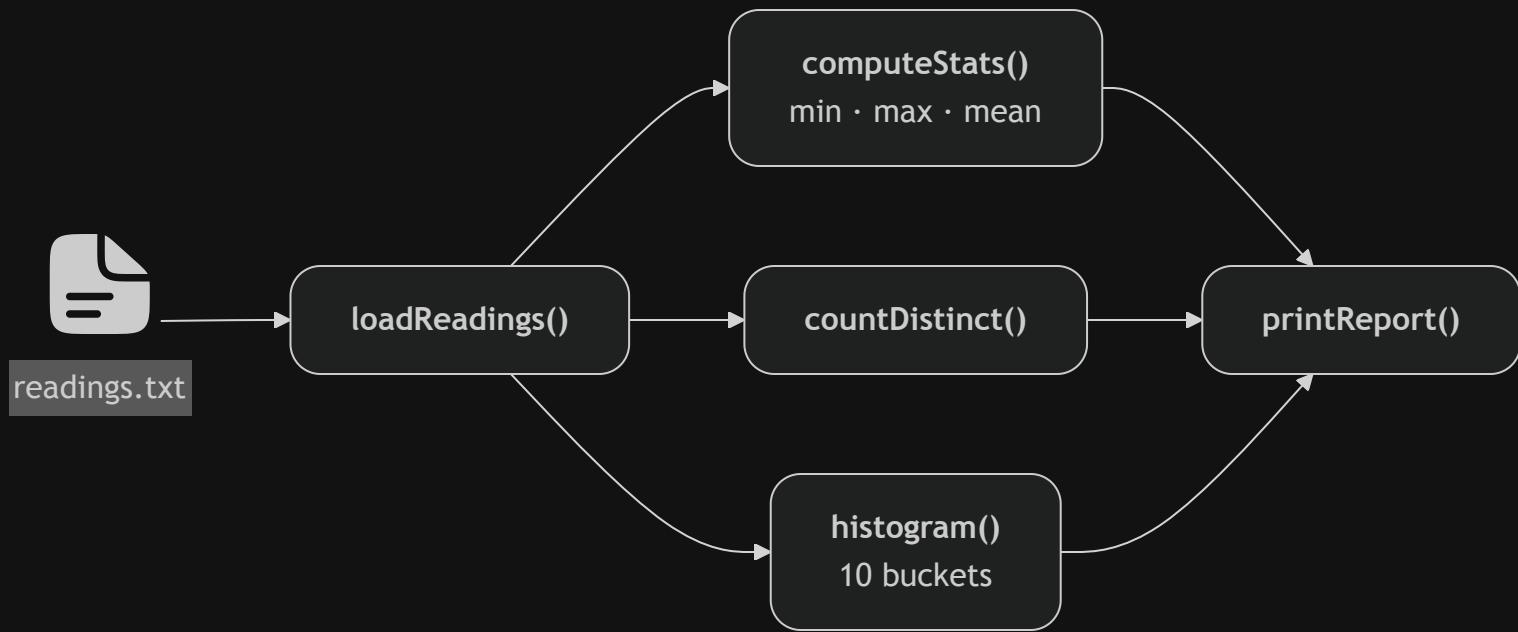
```
20 22 22 25 31 18 40 35 22 27
```

```
min 18 max 40 mean 26
```

```
distinct 8
```

```
0 1 6 2 1 0 0 0 0 0
```



Running the program

□ boundary.txt

95 99 100 97

Running the program

□ boundary.txt

```
95 99 100 97
```

```
min 95 max 100 mean 97
```

Running the program

□ boundary.txt

```
95 99 100 97
```

```
min 95 max 100 mean 97  
distinct 4
```

Running the program

□ boundary.txt

```
95 99 100 97
```

```
min 95 max 100 mean 97
```

```
distinct 4
```

```
0 0 0 0 0 0 0 0 0 3
```

Running the program

□ boundary.txt

```
95 99 100 97
```

```
min 95 max 100 mean 97  
distinct 4  
0 0 0 0 0 0 0 0 0 3
```

- Four readings went in, but the histogram shows three

Running the program

boundary.txt

```
95 99 100 97
```

```
min 95 max 100 mean 97  
distinct 4  
0 0 0 0 0 0 0 0 0 3
```

- Four readings went in, but the histogram shows three
- No warning, no error, exit code 0

Running the program

corrupt.txt

```
2147483000 2147483000 50
```

Running the program

📄 corrupt.txt

```
2147483000 2147483000 50
```

```
./humidity-stats corrupt.txt > out.txt
```

Running the program

📄 corrupt.txt

2147483000 2147483000 50

```
./humidity-stats corrupt.txt > out.txt
```

- `out.txt` is empty

Running the program

📄 corrupt.txt

```
2147483000 2147483000 50
```

```
./humidity-stats corrupt.txt > out.txt; echo $?  
138
```

- `out.txt` is empty
- Non-zero exit code

Running the program

```
./humidity-stats empty.txt
```

Running the program

```
./humidity-stats empty.txt
```

arm64 (Apple clang 21)

```
min 2147483647 max -2147483648 mean 0
```

exit code 0

Running the program

```
./humidity-stats empty.txt
```

arm64 (Apple clang 21)

min 2147483647 max -2147483648 mean 0

exit code 0

x86-64 (Clang 21.1)

Program terminated with signal: SIGFPE

exit code 136

Running the program

```
./humidity-stats empty.txt
```

arm64 (Apple clang 21)

```
min 2147483647 max -2147483648 mean 0
```

exit code 0

x86-64 (Clang 21.1)

```
Program terminated with signal: SIGFPE
```

exit code 136

Why is that allowed?

Undefined Behavior (UB)

"behavior for which this document imposes no requirements"

- C++ standard working draft, [defns.undefined]

"behavior for which this document imposes no requirements"

- C++ standard working draft, [defns.undefined]

"Permissible undefined behavior ranges from ignoring the situation completely with unpredictable results [...] to terminating a translation or execution"

Why is C++ like this?

Why is C++ like this?

- Zero-overhead principle: you don't pay for checks you didn't ask for

Why is C++ like this?

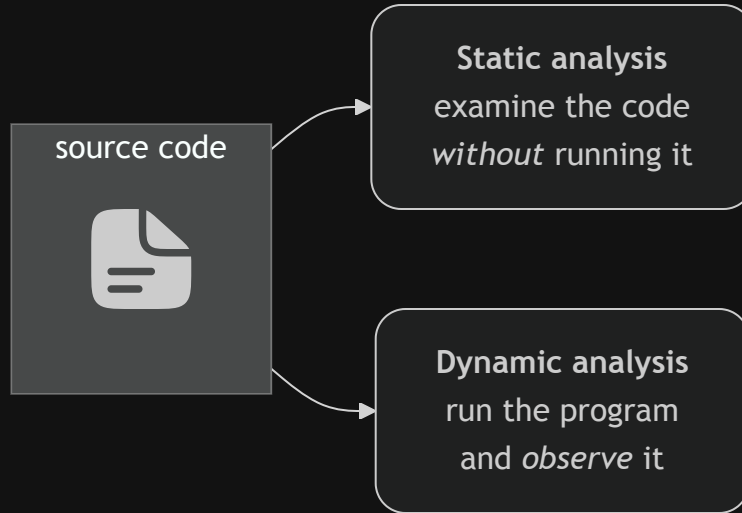
- Zero-overhead principle: you don't pay for checks you didn't ask for
- The compiler may assume undefined behavior never happens

Why is C++ like this?

- Zero-overhead principle: you don't pay for checks you didn't ask for
- The compiler may assume undefined behavior never happens
- That assumption is exactly why the runs behaved in different ways

Why is C++ like this?

- Zero-overhead principle: you don't pay for checks you didn't ask for
- The compiler may assume undefined behavior never happens
- That assumption is exactly why the runs behaved in different ways
- The language won't watch your program run, but tools can



No tool can answer everything

Rice's theorem (1953): Non-trivial questions about what a program will do are for the most part undecidable

No tool can answer everything

Rice's theorem (1953): Non-trivial questions about what a program will do are for the most part undecidable

- Analysis tools approximate the behavior of programs

No tool can answer everything

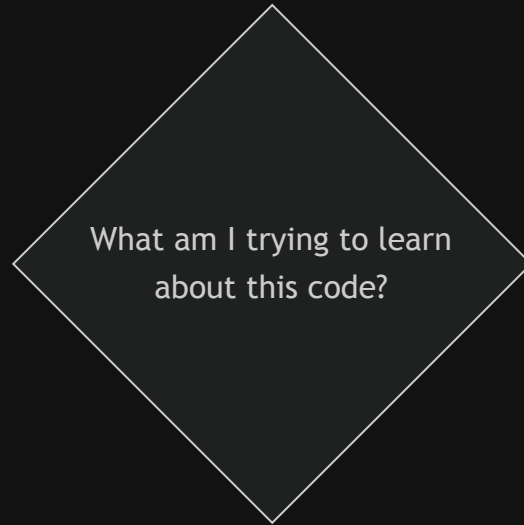
Rice's theorem (1953): Non-trivial questions about what a program will do are for the most part undecidable

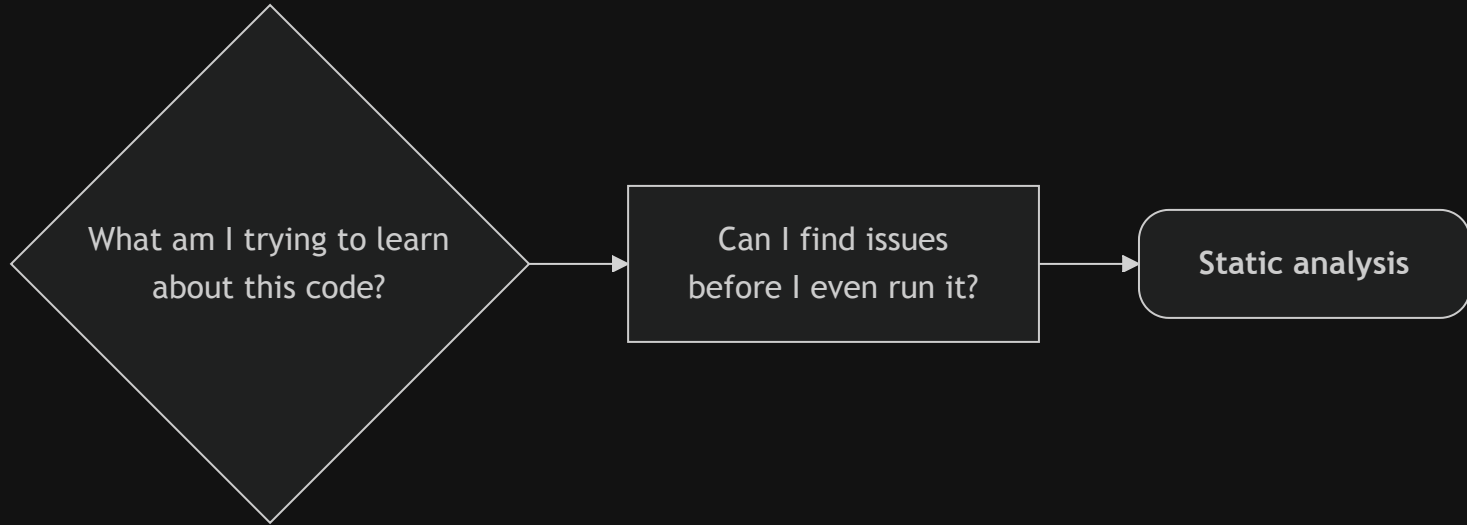
- Analysis tools approximate the behavior of programs
- False alarms and silent misses are inevitable

No tool can answer everything

Rice's theorem (1953): Non-trivial questions about what a program will do are for the most part undecidable

- Analysis tools approximate the behavior of programs
- False alarms and silent misses are inevitable
- The practical question becomes "what am I trying to learn?"





You already own a static analyzer

```
clang++ -std=c++20 -fsyntax-only stats.cpp main.cpp
```


You already own a static analyzer

```
clang++ -std=c++20 -fsyntax-only stats.cpp main.cpp
```

You already own a static analyzer

```
clang++ -std=c++20 -fsyntax-only stats.cpp main.cpp
```

- The compiler just parsed, type-checked, and resolved every line

You already own a static analyzer

```
clang++ -std=c++20 -fsyntax-only stats.cpp main.cpp
```

- The compiler just parsed, type-checked, and resolved every line
- It noticed plenty, it just wasn't asked to tell you

You already own a static analyzer

```
clang++ -std=c++20 -fsyntax-only stats.cpp main.cpp
```

- The compiler just parsed, type-checked, and resolved every line
- It noticed plenty, it just wasn't asked to tell you
- Diagnostics (beyond hard errors) are opt-in

You already own a static analyzer

C++ main.cpp

```
void printReport(const Stats& stats, bool verbose)
// ...
int checksum = 0;
printReport(computeStats(readings), false);
```

You already own a static analyzer

C++ main.cpp

```
void printReport(const Stats& stats, bool verbose)
// ...
int checksum = 0;
printReport(computeStats(readings), false);
```

```
clang++ -std=c++20 -fsyntax-only stats.cpp main.cpp
```

You already own a static analyzer

C++ main.cpp

```
void printReport(const Stats& stats, bool verbose)
// ...
int checksum = 0;
printReport(computeStats(readings), false);
```

```
clang++ -std=c++20 -Wall -fsyntax-only stats.cpp main.cpp
```

```
main.cpp:44:9: warning: unused variable 'checksum' [-Wunused-variable]
```

You already own a static analyzer

C++ main.cpp

```
void printReport(const Stats& stats, bool verbose)
// ...
int checksum = 0;
printReport(computeStats(readings), false);
```

```
clang++ -std=c++20 -Wall -Wextra -fsyntax-only stats.cpp main.cpp
```

```
main.cpp:44:9: warning: unused variable 'checksum' [-Wunused-variable]
main.cpp:29:43: warning: unused parameter 'verbose' [-Wunused-parameter]
```


You already own a static analyzer

C++ main.cpp

```
void printReport(const Stats& stats)
// ...
printReport(computeStats(readings));
```

```
clang++ -std=c++20 -Wall -Wextra -fsyntax-only stats.cpp main.cpp
```

```
main.cpp:44:9: warning: unused variable 'checksum' [-Wunused-variable]
main.cpp:29:43: warning: unused parameter 'verbose' [-Wunused-parameter]
```

You already own a static analyzer

C++ main.cpp

```
void printReport(const Stats& stats)
// ...
printReport(computeStats(readings));
```

```
clang++ -std=c++20 -Wall -Wextra -fsyntax-only stats.cpp main.cpp
```

```
main.cpp:44:9: warning: unused variable 'checksum' [-Wunused-variable]
main.cpp:29:43: warning: unused parameter 'verbose' [-Wunused-parameter]
```

- MSVC: `/W4` reports unused parameters (C4100) and initialized-but-unused locals (C4189)

Opting in further: `-Wconversion`

C++ stats.cpp

```
1  Stats computeStats(const std::vector<int>& readings)
2  {
3      // ...
4      int sum = // sum of readings
5      int count = readings.size();
6      int mean = sum / count;
7      return {minimum, maximum, mean};
8  }
```

Opting in further: `-Wconversion`

C++ stats.cpp

```
1  Stats computeStats(const std::vector<int>& readings)
2  {
3      // ...
4      int sum = // sum of readings
5      int count = readings.size();
6      int mean = sum / count;
7      return {minimum, maximum, mean};
8  }
```

Clang 21

```
warning: implicit conversion loses integer precision:
'size_type' (aka 'unsigned long') to 'int'
[-Wshorten-64-to-32]
```

Opting in further: `-Wconversion`

C++ stats.cpp

```
1  Stats computeStats(const std::vector<int>& readings)
2  {
3      // ...
4      int sum = // sum of readings
5      int count = readings.size();
6      int mean = sum / count;
7      return {minimum, maximum, mean};
8  }
```

Clang 21

```
warning: implicit conversion loses integer precision:
'size_type' (aka 'unsigned long') to 'int'
[-Wshorten-64-to-32]
```

GCC 16.1

```
warning: conversion from 'std::vector<int>::size_type'
{aka 'long unsigned int'} to 'int' may change value
[-Wconversion]
```

Opting in further: `-Wconversion`

C++ stats.cpp

```
1 Stats computeStats(const std::vector<int>& readings)
2 {
3     // ...
4     int sum = // sum of readings
5     int count = readings.size();
6     int mean = sum / count;
7     return {minimum, maximum, mean};
8 }
```

Clang 21

```
warning: implicit conversion loses integer precision:
'size_type' (aka 'unsigned long') to 'int'
[-Wshorten-64-to-32]
```

GCC 16.1

```
warning: conversion from 'std::vector<int>::size_type'
{aka 'long unsigned int'} to 'int' may change value
[-Wconversion]
```

- Same bug, different flag name and wording

Even the *same flag* differs

C++ main.cpp

```
1  std::array<int, 10> histogram(const std::vector<int>& readings)
2  {
3      std::array<int, 10> buckets{};
4      for (int r : readings)
5      {
6          ++buckets[r / 10];
7      }
8      return buckets;
9  }
```

Even the *same flag* differs

C++ main.cpp

```
1  std::array<int, 10> histogram(const std::vector<int>& readings)
2  {
3      std::array<int, 10> buckets{};
4      for (int r : readings)
5      {
6          ++buckets[r / 10];
7      }
8      return buckets;
9  }
```

Clang: **-Wconversion**

```
warning: implicit conversion changes signedness:
'int' to 'size_type' (aka 'unsigned long')
[-Wsign-conversion]
```


Even the *same flag* differs

C++ main.cpp

```
1  std::array<int, 10> histogram(const std::vector<int>& readings)
2  {
3      std::array<int, 10> buckets{};
4      for (int r : readings)
5      {
6          ++buckets[r / 10];
7      }
8      return buckets;
9  }
```

Clang: **-Wconversion**

```
warning: implicit conversion changes signedness:
'int' to 'size_type' (aka 'unsigned long')
[-Wsign-conversion]
```

GCC: needs **-Wsign-conversion** too

```
warning: conversion to 'std::array<int, 10>::size_type'
{aka 'long unsigned int'} from 'int' may change the
sign of the result [-Wsign-conversion]
```

Even the *same flag* differs

C++ main.cpp

```
1  std::array<int, 10> histogram(const std::vector<int>& readings)
2  {
3      std::array<int, 10> buckets{};
4      for (int r : readings)
5      {
6          ++buckets[r / 10];
7      }
8      return buckets;
9  }
```

Clang: `-Wconversion`

```
warning: implicit conversion changes signedness:
'int' to 'size_type' (aka 'unsigned long')
[-Wsign-conversion]
```

GCC: needs `-Wsign-conversion` too

```
warning: conversion to 'std::array<int, 10>::size_type'
{aka 'long unsigned int'} from 'int' may change the
sign of the result [-Wsign-conversion]
```

- Clang's `-Wconversion` implies `-Wsign-conversion`, GCC's does **not**

Evaluating the options

C++ main.cpp

```
for (int r : readings)
{
    ++buckets[r / 10];
}
```

Evaluating the options

C++ main.cpp

```
for (int r : readings)
{
    ++buckets[static_cast<std::size_t>(r / 10)];
}
```

- Casts state intent

Evaluating the options

C++ main.cpp

```
for (int r : readings)
{
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wsign-conversion"
    ++buckets[r / 10];
#pragma GCC diagnostic pop
}
```

- Casts state intent
- Pragmas can suppress the diagnostic

Evaluating the options

C++ main.cpp

```
for (int r : readings)
{
    ++buckets[r / 10];
}
```

- Casts state intent
- Pragmas can suppress the diagnostic
- We do neither

Linters

Reports suspicious patterns the compiler must accept (after C's `lint`, Bell Labs, 1978)

- `clang-tidy`
- `cppcheck`
- `include-what-you-use`

Clang-tidy

- Clang-based: parses your code with a real compiler frontend

Clang-tidy

- Clang-based: parses your code with a real compiler frontend
- Hundreds of checks in families: `bugprone-*`, `performance-*`, `modernize-*`, ...

Clang-tidy

- Clang-based: parses your code with a real compiler frontend
- Hundreds of checks in families: `bugprone-*`, `performance-*`, `modernize-*`, ...
- Configurable per-project via a `.clang-tidy` file

Clang-tidy

```
clang-tidy stats.cpp --checks='-* ,bugprone-*,performance-*,modernize-*' -- -std=c++20  
clang-tidy main.cpp --checks='-* ,bugprone-*,performance-*,modernize-*' -- -std=c++20
```

Clang-tidy

```
clang-tidy stats.cpp --checks='-*,bugprone-*,performance-*,modernize-*' -- -std=c++20  
clang-tidy main.cpp --checks='-*,bugprone-*,performance-*,modernize-*' -- -std=c++20
```

```
stats.cpp:19:17: warning: narrowing conversion from 'size_type' (aka 'unsigned long')  
to signed type 'int' is implementation-defined [bugprone-narrowing-conversions]
```

Clang-tidy

```
clang-tidy stats.cpp --checks='-*,bugprone-*,performance-*,modernize-*' -- -std=c++20  
clang-tidy main.cpp --checks='-*,bugprone-*,performance-*,modernize-*' -- -std=c++20
```

```
stats.cpp:19:17: warning: narrowing conversion from 'size_type' (aka 'unsigned long')  
to signed type 'int' is implementation-defined [bugprone-narrowing-conversions]
```

- The narrowing again: tools overlap

Clang-tidy

```
clang-tidy stats.cpp --checks='-*,bugprone-*,performance-*,modernize-*' -- -std=c++20  
clang-tidy main.cpp --checks='-*,bugprone-*,performance-*,modernize-*' -- -std=c++20
```

```
main.cpp:36:5: warning: an exception may be thrown in function 'main'  
which should not throw exceptions [bugprone-exception-escape]
```

Clang-tidy

```
clang-tidy stats.cpp --checks='-* ,bugprone-*,performance-*,modernize-*' -- -std=c++20  
clang-tidy main.cpp --checks='-* ,bugprone-*,performance-*,modernize-*' -- -std=c++20
```

```
main.cpp:36:5: warning: an exception may be thrown in function 'main'  
which should not throw exceptions [bugprone-exception-escape]
```

C++ main.cpp

```
1  std::vector<int>* loadReadings(const std::string& path);  
2  
3  int main(int argc, char* argv[])  
4  {  
5      // ...  
6      loadReadings(argv[1]);  
7  }
```

Clang-tidy

```
clang-tidy stats.cpp --checks='-*,bugprone-*,performance-*,modernize-*' -- -std=c++20
clang-tidy main.cpp --checks='-*,bugprone-*,performance-*,modernize-*' -- -std=c++20
```

```
main.cpp:36:5: warning: an exception may be thrown in function 'main'
which should not throw exceptions [bugprone-exception-escape]
```

C++ main.cpp

```
1  std::vector<int>* loadReadings(const std::string& path);
2
3  int main(int argc, char* argv[])
4  {
5      // ...
6      loadReadings(argv[1]);
7  }
```


The same run also said...

```
stats.cpp:6:1: warning: use a trailing return type for this function  
[modernize-use-trailing-return-type]
```

The same run also said...

```
stats.cpp:6:1: warning: use a trailing return type for this function  
[modernize-use-trailing-return-type]
```

Seven times.

The same run also said...

```
stats.cpp:6:1: warning: use a trailing return type for this function  
[modernize-use-trailing-return-type]
```

Seven times. Plus:

```
873 warnings generated.  
Suppressed 867 warnings (867 in non-user code).
```

The same run also said...

```
stats.cpp:6:1: warning: use a trailing return type for this function  
[modernize-use-trailing-return-type]
```

Seven times. Plus:

```
873 warnings generated.  
Suppressed 867 warnings (867 in non-user code).
```

- Style checks are often opinions

The same run also said...

```
stats.cpp:6:1: warning: use a trailing return type for this function  
[modernize-use-trailing-return-type]
```

Seven times. Plus:

```
873 warnings generated.  
Suppressed 867 warnings (867 in non-user code).
```

- Style checks are often opinions
- Check sets and signal-to-noise are your responsibility

Different tools, different misses

C++ clamp_bug.cpp

```
1 // Clamp a raw humidity reading into the valid range [0, 100].
2 int clampReading(int reading)
3 {
4     if (reading > 100)
5     {
6         if (reading < 0)
7         {
8             return 0;
9         }
10        return 100;
11    }
12    return reading;
13 }
```

Different tools, different misses

C++ clamp_bug.cpp

```
1  // Clamp a raw humidity reading into the valid range [0, 100].
2  int clampReading(int reading)
3  {
4      if (reading > 100)
5      {
6          if (reading < 0)
7          {
8              return 0;
9          }
10         return 100;
11     }
12     return reading;
13 }
```

Different tools, different misses

Tool

Dead code reported

Different tools, different misses

Tool

Dead code reported

clang `-Wall -Wextra -Wconversion`

Different tools, different misses

Tool

Dead code reported

clang `-Wall -Wextra -Wconversion`



Different tools, different misses

Tool	Dead code reported
clang <code>-Wall -Wextra -Wconversion</code>	
GCC 16.1 <code>-Wall -Wextra</code>	

Different tools, different misses

Tool	Dead code reported
clang <code>-Wall -Wextra -Wconversion</code>	
GCC 16.1 <code>-Wall -Wextra</code>	

Different tools, different misses

Tool	Dead code reported
clang <code>-Wall -Wextra -Wconversion</code>	
GCC 16.1 <code>-Wall -Wextra</code>	
clang-tidy <code>bugprone-*</code> + <code>clang-analyzer-*</code>	

Different tools, different misses

Tool	Dead code reported
clang <code>-Wall -Wextra -Wconversion</code>	
GCC 16.1 <code>-Wall -Wextra</code>	
clang-tidy <code>bugprone-*</code> + <code>clang-analyzer-*</code>	

Different tools, different misses

Tool	Dead code reported
clang <code>-Wall -Wextra -Wconversion</code>	
GCC 16.1 <code>-Wall -Wextra</code>	
clang-tidy <code>bugprone-*</code> + <code>clang-analyzer-*</code>	
cppcheck <code>--enable=all</code>	

Different tools, different misses

Tool	Dead code reported
clang <code>-Wall -Wextra -Wconversion</code>	
GCC 16.1 <code>-Wall -Wextra</code>	
clang-tidy <code>bugprone-*</code> + <code>clang-analyzer-*</code>	
cppcheck <code>--enable=all</code>	
<code>clamp_bug.cpp:6:21: warning: Opposite inner 'if' condition leads to a dead code block. [oppositeInnerCondition]</code>	

Different tools, different misses

Tool	Dead code reported
clang <code>-Wall -Wextra -Wconversion</code>	
GCC 16.1 <code>-Wall -Wextra</code>	
clang-tidy <code>bugprone-*</code> + <code>clang-analyzer-*</code>	
cppcheck <code>--enable=all</code>	
<code>clamp_bug.cpp:6:21: warning: Opposite inner 'if' condition leads to a dead code block. [oppositeInnerCondition]</code>	

- Standalone, value-flow analysis notices the two conditions contradict each other

The dependencies nobody reviews

Includes are also a matter of code analysis:

The dependencies nobody reviews

Includes are also a matter of code analysis:

- Every `#include` is a **transitive** dependency declaration

The dependencies nobody reviews

Includes are also a matter of code analysis:

- Every `#include` is a **transitive** dependency declaration
- Code that compiles "by luck" is fragile

The dependencies nobody reviews

Includes are also a matter of code analysis:

- Every `#include` is a **transitive** dependency declaration
- Code that compiles "by luck" is fragile
- Dead includes silently tax every rebuild

include-what-you-use

```
// stats.hpp
#include <string>
#include <vector>
// main.cpp
#include <array>
#include <fstream>
#include <iostream>
```

```
include-what-you-use -std=c++20 stats.cpp
include-what-you-use -std=c++20 main.cpp
```

include-what-you-use

```
// stats.hpp
#include <string>
#include <vector>
// main.cpp
#include <array>
#include <fstream>
#include <iostream>
```

```
include-what-you-use -std=c++20 stats.cpp
include-what-you-use -std=c++20 main.cpp
```

stats.hpp should remove these lines:
- #include <string> // lines 3-3

main.cpp should add these lines:
#include <string> // for basic_string, string
#include <vector> // for vector

include-what-you-use

```
// stats.hpp
#include <string>
#include <vector>
// main.cpp
#include <array>
#include <fstream>
#include <iostream>
```

```
include-what-you-use -std=c++20 stats.cpp
include-what-you-use -std=c++20 main.cpp
```

stats.hpp should remove these lines:
- #include <string> // lines 3-3

main.cpp should add these lines:
#include <string> // for basic_string, string
#include <vector> // for vector

- `<string>` is dead weight in `stats.hpp`

include-what-you-use

```
// stats.hpp
#include <string>
#include <vector>
// main.cpp
#include <array>
#include <fstream>
#include <iostream>
```

```
include-what-you-use -std=c++20 stats.cpp
include-what-you-use -std=c++20 main.cpp
```

stats.hpp should remove these lines:
- #include <string> // lines 3-3

main.cpp should add these lines:
#include <string> // for basic_string, string
#include <vector> // for vector

- `<string>` is dead weight in `stats.hpp`
- But `main.cpp` used `std::string` and `std::vector` *through* it

include-what-you-use

```
// stats.hpp
#include <vector>
// main.cpp
#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>
```

```
include-what-you-use -std=c++20 stats.cpp
include-what-you-use -std=c++20 main.cpp
```

stats.hpp should remove these lines:

```
- #include <string> // lines 3-3
```

main.cpp should add these lines:

```
#include <string>    // for basic_string, string
#include <vector>    // for vector
```

Static analysis

The tools checked all inputs and all paths.

Static analysis

The tools checked all inputs and all paths.

Static analysis

The tools checked all inputs and all paths. And yet...

Static analysis

The tools checked all inputs and all paths. And yet...

- A reading vanishes

```
./humidity-stats boundary.txt → 0 0 0 0 0 0 0 0 0 3 (exit 0)
```

Static analysis

The tools checked all inputs and all paths. And yet...

- A reading vanishes

```
./humidity-stats boundary.txt → 0 0 0 0 0 0 0 0 0 3 (exit 0)
```

- A crash with lost output

```
./humidity-stats corrupt.txt → output lost in a crash (exit 138)
```

Static analysis

The tools checked all inputs and all paths. And yet...

- A reading vanishes

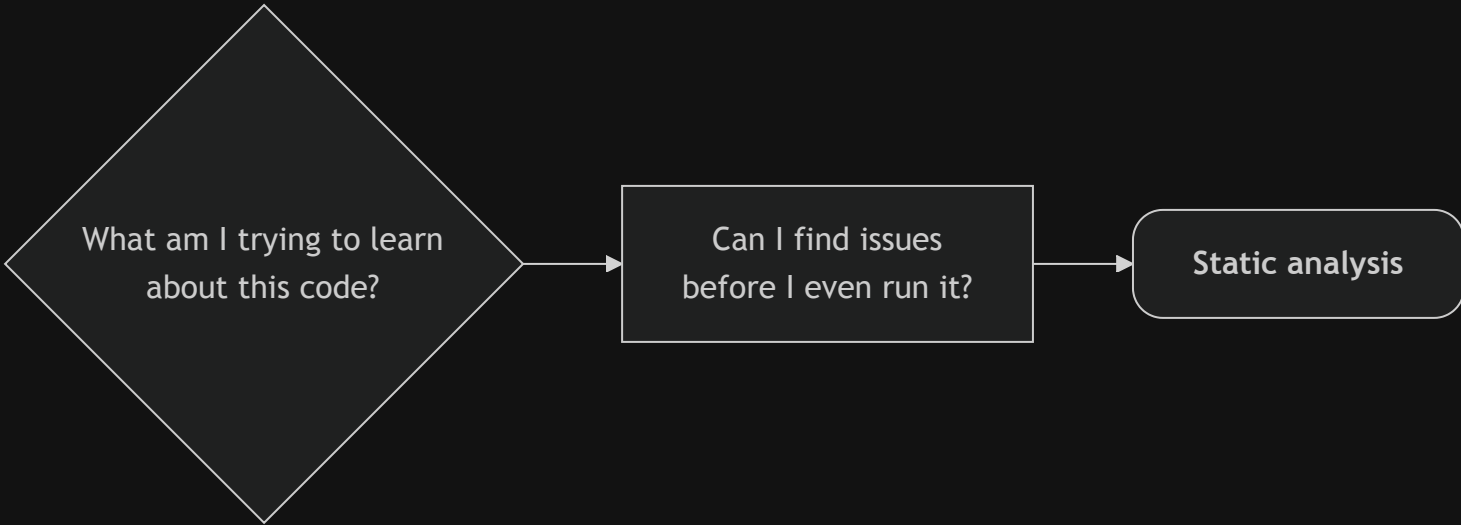
```
./humidity-stats boundary.txt → 0 0 0 0 0 0 0 0 0 3 (exit 0)
```

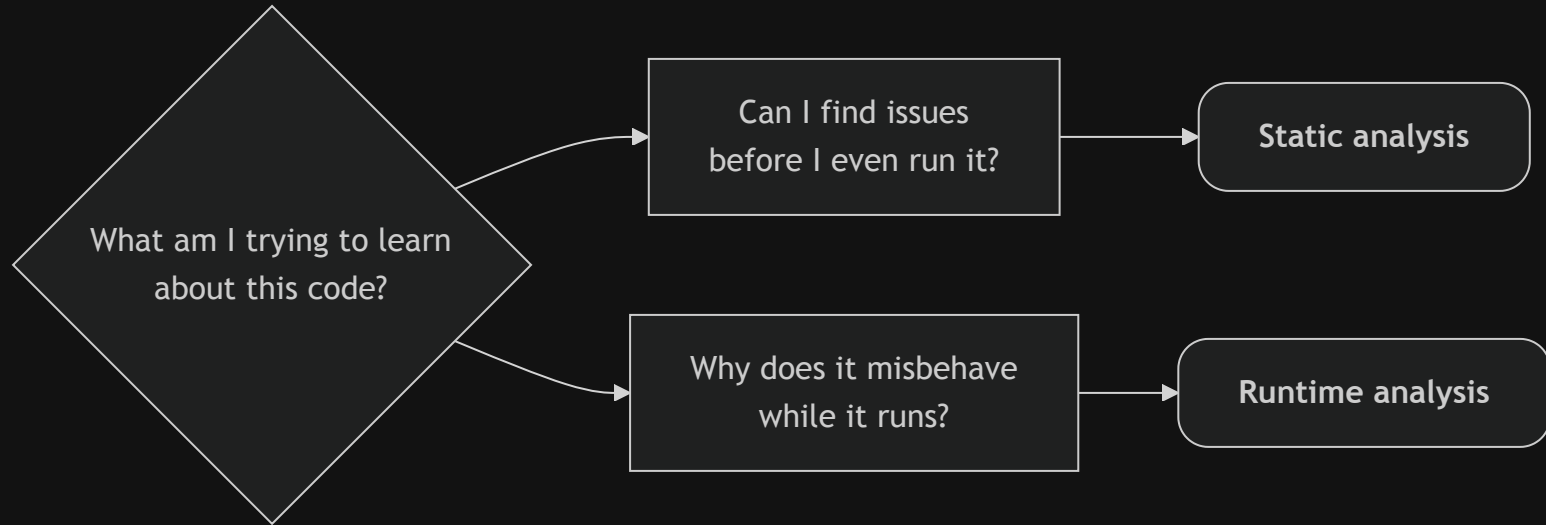
- A crash with lost output

```
./humidity-stats corrupt.txt → output lost in a crash (exit 138)
```

- Garbage with no error

```
./humidity-stats empty.txt → min 2147483647 max -2147483648 (exit 0)
```



Start with the tool you already have

A debugger + debug build (`-O0 -g`):

```
1  lldb -- ./humidity-stats corrupt.txt
```

```
(lldb) run
```

Start with the tool you already have

A debugger + debug build (`-O0 -g`):

```
1  lldb -- ./humidity-stats corrupt.txt
```

```
(lldb) run
stop reason = EXC_BAD_ACCESS (code=2, address=0x1a31318b8)
  frame #0: histogram(readings=size=3) at main.cpp:26
→   26      ++buckets[r / 10];
```

Start with the tool you already have

A debugger + debug build (`-O0 -g`):

```
1  lldb -- ./humidity-stats corrupt.txt
```

```
(lldb) run
stop reason = EXC_BAD_ACCESS (code=2, address=0x1a31318b8)
frame #0: histogram(readings=size=3) at main.cpp:26
→ 26      ++buckets[r / 10];
```

C++ main.cpp

```
1  std::array<int, 10> histogram(const std::vector<int>& readings)
2  {
3      std::array<int, 10> buckets{};
4      for (int r : readings)
5      {
6          ++buckets[r / 10];
7      }
8      return buckets;
9  }
```

Start with the tool you already have

A debugger + debug build (`-O0 -g`):

```
1  lldb -- ./humidity-stats corrupt.txt
```

```
(lldb) run
stop reason = EXC_BAD_ACCESS (code=2, address=0x1a31318b8)
  frame #0: histogram(readings=size=3) at main.cpp:26
→   26      ++buckets[r / 10];
(lldb) print r
```

Start with the tool you already have

A debugger + debug build (`-O0 -g`):

```
1  lldb -- ./humidity-stats corrupt.txt
```

```
(lldb) run
stop reason = EXC_BAD_ACCESS (code=2, address=0x1a31318b8)
  frame #0: histogram(readings=size=3) at main.cpp:26
→   26      ++buckets[r / 10];
(lldb) print r
(int) 2147483000
```

Start with the tool you already have

A debugger + debug build (`-O0 -g`):

```
1  lldb -- ./humidity-stats corrupt.txt
```

```
(lldb) run
stop reason = EXC_BAD_ACCESS (code=2, address=0x1a31318b8)
  frame #0: histogram(readings=size=3) at main.cpp:26
→   26      ++buckets[r / 10];
(lldb) print r
(int) 2147483000
(lldb) print buckets
```


Start with the tool you already have

A debugger + debug build (`-O0 -g`):

```
1  lldb -- ./humidity-stats corrupt.txt
```

```
(lldb) run
stop reason = EXC_BAD_ACCESS (code=2, address=0x1a31318b8)
  frame #0: histogram(readings=size=3) at main.cpp:26
→   26      ++buckets[r / 10];
(lldb) print r
(int) 2147483000
(lldb) print buckets
(std::array<int, 10>) { _M_elems = ([0] = 0, ..., [9] = 0) }
```

Start with the tool you already have

A debugger + debug build (`-O0 -g`):

```
1  lldb -- ./humidity-stats corrupt.txt
```

```
(lldb) run
stop reason = EXC_BAD_ACCESS (code=2, address=0x1a31318b8)
  frame #0: histogram(readings=size=3) at main.cpp:26
→   26      ++buckets[r / 10];
(lldb) print r
(int) 2147483000
(lldb) print buckets
(std::array<int, 10>) { _M_elems = ([0] = 0, ..., [9] = 0) }
```

The fix: garbage stays out of the histogram

C++ main.cpp

```
1  std::array<int, 10> buckets{};
2  for (int r : readings)
3  {
4      ++buckets[r / 10];
5  }
```

The fix: garbage stays out of the histogram

C++ main.cpp

```
1  std::array<int, 10> buckets{};
2  for (int r : readings)
3  {
4      if (r < 0 || r > 100)
5      {
6          continue;
7      }
8      ++buckets[r / 10];
9  }
```

The fix: garbage stays out of the histogram

C++ main.cpp

```
1  std::array<int, 10> buckets{};
2  for (int r : readings)
3  {
4      if (r < 0 || r > 100)
5      {
6          continue;
7      }
8      ++buckets[r / 10];
9  }
```

```
./humidity-stats corrupt.txt      # 2147483000 2147483000 50
0 0 0 0 0 1 0 0 0 0              (outside the documented range: discarded)
```

The fix: garbage stays out of the histogram

C++ main.cpp

```
1  std::array<int, 10> buckets{};
2  for (int r : readings)
3  {
4      if (r < 0 || r > 100)
5      {
6          continue;
7      }
8      ++buckets[r / 10];
9  }
```

```
./humidity-stats corrupt.txt      # 2147483000 2147483000 50
0 0 0 0 0 1 0 0 0 0              (outside the documented range: discarded)
```

```
./humidity-stats boundary.txt     # 95 99 100 97
0 0 0 0 0 0 0 0 0 3              (four readings in, three counted, exit 0)
```

Sanitizers

Compiler instrumentation + a runtime library

Sanitizers

Compiler instrumentation + a runtime library

- The compiler injects checks

Sanitizers

Compiler instrumentation + a runtime library

- The compiler injects checks
 - Memory accesses

Sanitizers

Compiler instrumentation + a runtime library

- The compiler injects checks
 - Memory accesses
 - Arithmetic

Sanitizers

Compiler instrumentation + a runtime library

- The compiler injects checks
 - Memory accesses
 - Arithmetic
 - Threads

Sanitizers

Compiler instrumentation + a runtime library

- The compiler injects checks
 - Memory accesses
 - Arithmetic
 - Threads
- A runtime library tracks extra state

Sanitizers

Compiler instrumentation + a runtime library

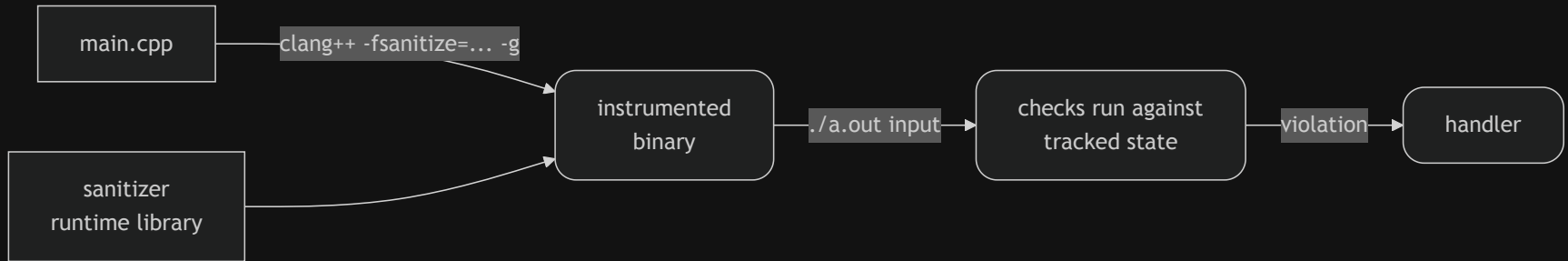
- The compiler injects checks
 - Memory accesses
 - Arithmetic
 - Threads
- A runtime library tracks extra state
- On a violation: handle the error

Sanitizers

Compiler instrumentation + a runtime library

- The compiler injects checks
 - Memory accesses
 - Arithmetic
 - Threads
- A runtime library tracks extra state
- On a violation: handle the error
 - By default: print a report and abort

How a sanitized build works



ASan: Address Sanitizer

```
g++ -std=c++20 -g -O0 -fsanitize=address stats.cpp main.cpp -o ss-asan  
./ss-asan boundary.txt
```


ASan: Address Sanitizer

```
g++ -std=c++20 -g -O0 -fsanitize=address stats.cpp main.cpp -o ss-asan  
./ss-asan boundary.txt
```

```
=ERROR: AddressSanitizer: stack-buffer-overflow  
READ of size 4 at 0x... thread T0  
    #0 in histogram(std::vector<int> const&) main.cpp:30  
    #1 in main main.cpp:60  
...  
SUMMARY: AddressSanitizer: stack-buffer-overflow  
    main.cpp:30 in histogram( ... )
```

ASan: Address Sanitizer

```
g++ -std=c++20 -g -O0 -fsanitize=address stats.cpp main.cpp -o ss-asan  
./ss-asan boundary.txt
```

```
=ERROR: AddressSanitizer: stack-buffer-overflow  
READ of size 4 at 0x... thread T0  
    #0 in histogram(std::vector<int> const&) main.cpp:30  
    #1 in main main.cpp:60  
...  
SUMMARY: AddressSanitizer: stack-buffer-overflow  
    main.cpp:30 in histogram(...)
```

- The error class

ASan: Address Sanitizer

```
g++ -std=c++20 -g -O0 -fsanitize=address stats.cpp main.cpp -o ss-asan  
./ss-asan boundary.txt
```

```
=ERROR: AddressSanitizer: stack-buffer-overflow  
READ of size 4 at 0x... thread T0  
    #0 in histogram(std::vector<int> const&) main.cpp:30  
    #1 in main main.cpp:60  
***  
SUMMARY: AddressSanitizer: stack-buffer-overflow  
    main.cpp:30 in histogram(...)
```

- The error class
- The call stack that got there

ASan fix

C++ main.cpp

```
1  if (r < 0 || r > 100)
2  {
3      continue;
4  }
5  ++buckets[r / 10];
```

ASan fix

C++ main.cpp

```
1  if (r < 0 || r > 100)
2  {
3      continue;
4  }
5  ++buckets[std::min(r / 10, 9)];
```

ASan fix

C++ main.cpp

```
1  if (r < 0 || r > 100)
2  {
3      continue;
4  }
5  ++buckets[std::min(r / 10, 9)];
```

```
./humidity-stats boundary.txt    # 95 99 100 97
0 0 0 0 0 0 0 0 0 4
```

UBSan: Undefined Behavior Sanitizer

`corrupt.txt` no longer crashes:

```
./humidity-stats corrupt.txt      # 2147483000 2147483000 50  
min 50 max 2147483000 mean -415
```

UBSan: Undefined Behavior Sanitizer

`corrupt.txt` no longer crashes:

```
./humidity-stats corrupt.txt      # 2147483000 2147483000 50  
min 50 max 2147483000 mean -415
```

`-fsanitize=undefined` :

```
stats.cpp:17:13: runtime error: signed integer overflow:  
2147483000 + 2147483000 cannot be represented in type 'int'
```


UBSan: Undefined Behavior Sanitizer

`corrupt.txt` no longer crashes:

```
./humidity-stats corrupt.txt      # 2147483000 2147483000 50  
min 50 max 2147483000 mean -415
```

`-fsanitize=undefined`:

```
stats.cpp:17:13: runtime error: signed integer overflow:  
2147483000 + 2147483000 cannot be represented in type 'int'
```

 stats.cpp

```
1  int sum = 0;  
2  
3  for (int r : readings)  
4  {  
5      sum += r;  
6  }
```

UBSan: Undefined Behavior Sanitizer

`corrupt.txt` no longer crashes:

```
./humidity-stats corrupt.txt      # 2147483000 2147483000 50  
min 50 max 2147483000 mean -415
```

`-fsanitize=undefined` :

```
stats.cpp:17:13: runtime error: signed integer overflow:  
2147483000 + 2147483000 cannot be represented in type 'int'
```

 stats.cpp

```
1  int sum = 0;  
2  
3  for (int r : readings)  
4  {  
5      sum += r;  
6  }
```

- No compiler warned: the compiler may *assume* UB never happens.

The fix: a type that states the intent

C++ stats.cpp

```
1  int sum = 0;
2
3  for (int r : readings)
4  {
5      sum += r;
6  }
```

The fix: a type that states the intent

C++ stats.cpp

```
1  int sum = 0;
2
3  for (int r : readings)
4  {
5      sum += r;
6  }
```

The fix: a type that states the intent

C++ stats.cpp

```
1  std::int64_t sum = 0;
2
3  for (int r : readings)
4  {
5      sum += r;
6  }
```

The fix: a type that states the intent

C++ stats.cpp

```
1  std::int64_t sum = 0;
2
3  for (int r : readings)
4  {
5      sum += r;
6  }
```

```
min 50 max 2147483000 mean 1431655350
```

LSan: Leak Sanitizer

C++ main.cpp

```
1  std::vector<int>* loadReadings(const std::string& path)
2  {
3      auto* readings = new std::vector<int>;
4      std::ifstream in(path);
5      int value = 0;
6      while (in >> value)
7      {
8          readings->push_back(value);
9      }
10     return readings;
11 }
12 // ... in main():
13 const std::vector<int>& readings = *loadReadings(argv[1]);
```

LSan: Leak Sanitizer

C++ main.cpp

```
1  std::vector<int>* loadReadings(const std::string& path)
2  {
3      auto* readings = new std::vector<int>;
4      std::ifstream in(path);
5      int value = 0;
6      while (in >> value)
7      {
8          readings->push_back(value);
9      }
10     return readings;
11 }
12 // ... in main():
13 const std::vector<int>& readings = *loadReadings(argv[1]);
```


LSan: Leak Sanitizer

C++ main.cpp

```
1  std::vector<int>* loadReadings(const std::string& path)
2  {
3      auto* readings = new std::vector<int>;
4      std::ifstream in(path);
5      int value = 0;
6      while (in >> value)
7      {
8          readings->push_back(value);
9      }
10     return readings;
11 }
12 // ... in main():
13 const std::vector<int>& readings = *loadReadings(argv[1]);
```

Leak Sanitizer report

```
clang++ -std=c++20 -g -O0 -fsanitize=leak stats.cpp main.cpp -o ss-lsan  
./ss-lsan readings.txt
```

```
=ERROR: LeakSanitizer: detected memory leaks  
Direct leak of 24 byte(s) in 1 object(s) allocated from:  
    #1 in loadReadings( ... ) main.cpp:12  
    #2 in main main.cpp:51  
Indirect leak of 64 byte(s) in 1 object(s) allocated from:  
    ... (the vector's heap buffer)  
SUMMARY: LeakSanitizer: 88 byte(s) leaked in 2 allocation(s).
```

Leak Sanitizer report

```
clang++ -std=c++20 -g -O0 -fsanitize=leak stats.cpp main.cpp -o ss-lsan
./ss-lsan readings.txt
```

```
=ERROR: LeakSanitizer: detected memory leaks
Direct leak of 24 byte(s) in 1 object(s) allocated from:
    #1 in loadReadings( ... ) main.cpp:12
    #2 in main main.cpp:51
Indirect leak of 64 byte(s) in 1 object(s) allocated from:
    ... (the vector's heap buffer)
SUMMARY: LeakSanitizer: 88 byte(s) leaked in 2 allocation(s).
```

- Direct leak: the `std::vector` object itself

Leak Sanitizer report

```
clang++ -std=c++20 -g -O0 -fsanitize=leak stats.cpp main.cpp -o ss-lsan
./ss-lsan readings.txt
```

```
=ERROR: LeakSanitizer: detected memory leaks
Direct leak of 24 byte(s) in 1 object(s) allocated from:
    #1 in loadReadings( ...) main.cpp:12
    #2 in main main.cpp:51
Indirect leak of 64 byte(s) in 1 object(s) allocated from:
    ... (the vector's heap buffer)
SUMMARY: LeakSanitizer: 88 byte(s) leaked in 2 allocation(s).
```

- Direct leak: the `std::vector` object itself
- Indirect leak: the vector's heap buffer

Leak Sanitizer report

```
clang++ -std=c++20 -g -O0 -fsanitize=leak stats.cpp main.cpp -o ss-lsan  
./ss-lsan readings.txt
```

```
=ERROR: LeakSanitizer: detected memory leaks  
Direct leak of 24 byte(s) in 1 object(s) allocated from:  
    #1 in loadReadings( ...) main.cpp:12  
    #2 in main main.cpp:51  
Indirect leak of 64 byte(s) in 1 object(s) allocated from:  
    ... (the vector's heap buffer)  
SUMMARY: LeakSanitizer: 88 byte(s) leaked in 2 allocation(s).
```

- Direct leak: the `std::vector` object itself
- Indirect leak: the vector's heap buffer
- 88 bytes in 2 allocations

Leak Sanitizer report

C++ main.cpp

```
1 std::vector<int>* loadReadings(const std::string& path)
2 {
3     auto* readings = new std::vector<int>;
4     // ... read the file ...
5     return readings;
6 }
7 // ... in main():
8 const std::vector<int>& readings = *loadReadings(argv[1]);
```

Leak Sanitizer report

C++ main.cpp

```
1  std::vector<int> loadReadings(const std::string& path)
2  {
3      std::vector<int> readings;
4      // ... read the file ...
5      return readings;
6  }
7  // ... in main():
8  const std::vector<int> readings = loadReadings(argv[1]);
```

TSan: Thread Sanitizer

Suppose a colleague "sped up" the sum:

C++ stats.cpp

```
1  int sum = 0;
2  auto worker = [&](std::size_t lo, std::size_t hi)
3  {
4      for (std::size_t i = lo; i < hi; ++i)
5      {
6          sum += readings[i];
7      }
8  };
9
10 const std::size_t mid = readings.size() / 2;
11 std::thread t1(worker, 0, mid);
12 std::thread t2(worker, mid, readings.size());
```


TSan: Thread Sanitizer

Suppose a colleague "sped up" the sum:

C++ stats.cpp

```
1  int sum = 0;
2  auto worker = [&](std::size_t lo, std::size_t hi)
3  {
4      for (std::size_t i = lo; i < hi; ++i)
5      {
6          sum += readings[i];
7      }
8  };
9
10 const std::size_t mid = readings.size() / 2;
11 std::thread t1(worker, 0, mid);
12 std::thread t2(worker, mid, readings.size());
```

TSan: Thread Sanitizer

Suppose a colleague "sped up" the sum:

C++ stats.cpp

```
1  int sum = 0;
2  auto worker = [&](std::size_t lo, std::size_t hi)
3  {
4      for (std::size_t i = lo; i < hi; ++i)
5      {
6          sum += readings[i];
7      }
8  };
9
10 const std::size_t mid = readings.size() / 2;
11 std::thread t1(worker, 0, mid);
12 std::thread t2(worker, mid, readings.size());
```

- Shared accumulator, no lock

TSan: Thread Sanitizer

Suppose a colleague "sped up" the sum:

C++ stats.cpp

```
1  int sum = 0;
2  auto worker = [&](std::size_t lo, std::size_t hi)
3  {
4      for (std::size_t i = lo; i < hi; ++i)
5      {
6          sum += readings[i];
7      }
8  };
9
10 const std::size_t mid = readings.size() / 2;
11 std::thread t1(worker, 0, mid);
12 std::thread t2(worker, mid, readings.size());
```

- Shared accumulator, no lock
- Half the time, it works every time

Two threads, one unguarded write

```
clang++ -std=c++20 -g -O0 -fsanitize=thread stats.cpp main.cpp -o ss-tsan  
./ss-tsan readings.txt
```

```
WARNING: ThreadSanitizer: data race  
  Write of size 4 at 0x... by thread T2:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
Previous write of size 4 at 0x... by thread T1:  
  #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
Thread T2 (running) created by main thread at: ... stats.cpp:31  
Thread T1 (finished) created by main thread at: ... stats.cpp:30
```

Two threads, one unguarded write

```
clang++ -std=c++20 -g -O0 -fsanitize=thread stats.cpp main.cpp -o ss-tsan  
./ss-tsan readings.txt
```

```
WARNING: ThreadSanitizer: data race  
  Write of size 4 at 0x... by thread T2:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
  Previous write of size 4 at 0x... by thread T1:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
  Thread T2 (running) created by main thread at: ... stats.cpp:31  
  Thread T1 (finished) created by main thread at: ... stats.cpp:30
```

- Race reported by TSan

Two threads, one unguarded write

```
clang++ -std=c++20 -g -O0 -fsanitize=thread stats.cpp main.cpp -o ss-tsan  
./ss-tsan readings.txt
```

```
WARNING: ThreadSanitizer: data race  
  Write of size 4 at 0x... by thread T2:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
  Previous write of size 4 at 0x... by thread T1:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
  Thread T2 (running) created by main thread at: ... stats.cpp:31  
  Thread T1 (finished) created by main thread at: ... stats.cpp:30
```

- Race reported by TSan
- Both conflicting accesses

Two threads, one unguarded write

```
clang++ -std=c++20 -g -O0 -fsanitize=thread stats.cpp main.cpp -o ss-tsan  
./ss-tsan readings.txt
```

```
WARNING: ThreadSanitizer: data race  
  Write of size 4 at 0x... by thread T2:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
  Previous write of size 4 at 0x... by thread T1:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
  Thread T2 (running) created by main thread at: ... stats.cpp:31  
  Thread T1 (finished) created by main thread at: ... stats.cpp:30
```

- Race reported by TSan
- Both conflicting accesses
- Where each thread was created

Two threads, one unguarded write

```
clang++ -std=c++20 -g -O0 -fsanitize=thread stats.cpp main.cpp -o ss-tsan  
./ss-tsan readings.txt
```

```
WARNING: ThreadSanitizer: data race  
  Write of size 4 at 0x... by thread T2:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
  Previous write of size 4 at 0x... by thread T1:  
    #0 computeStats( ... )::operator()( ... ) stats.cpp:27  
  Thread T2 (running) created by main thread at: ... stats.cpp:31  
  Thread T1 (finished) created by main thread at: ... stats.cpp:30
```

- Race reported by TSan
- Both conflicting accesses
- Where each thread was created
- The fix: Use `std::atomic<int> sum`

What it costs

Tool	Typical slowdown
ASan	~2×
TSan	5-15× (and 5-10× memory)
UBSan	low, check-dependent
LSan	~none (checks at exit)

Can I turn them all on at once?

	ASan	LSan	TSan	UBSan
ASan	-	✓	✗	✓
LSan	✓	-	✗	✓
TSan	✗	✗	-	✓
UBSan	✓	✓	✓	-

Can I turn them all on at once?

	ASan	LSan	TSan	UBSan
ASan	-	✓	✗	✓
LSan	✓	-	✗	✓
TSan	✗	✗	-	✓
UBSan	✓	✓	✓	-

- UBSan stacks with everything

Can I turn them all on at once?

	ASan	LSan	TSan	UBSan
ASan	-	✓	✗	✓
LSan	✓	-	✗	✓
TSan	✗	✗	-	✓
UBSan	✓	✓	✓	-

- UBSan stacks with everything
- Shadow-memory clashes (ASan, TSan)

Can I turn them all on at once?

	ASan	LSan	TSan	UBSan
ASan	-	✓	✗	✓
LSan	✓	-	✗	✓
TSan	✗	✗	-	✓
UBSan	✓	✓	✓	-

- UBSan stacks with everything
- Shadow-memory clashes (ASan, TSan)
- LSan is unavailable on Apple Silicon

Can I turn them all on at once?

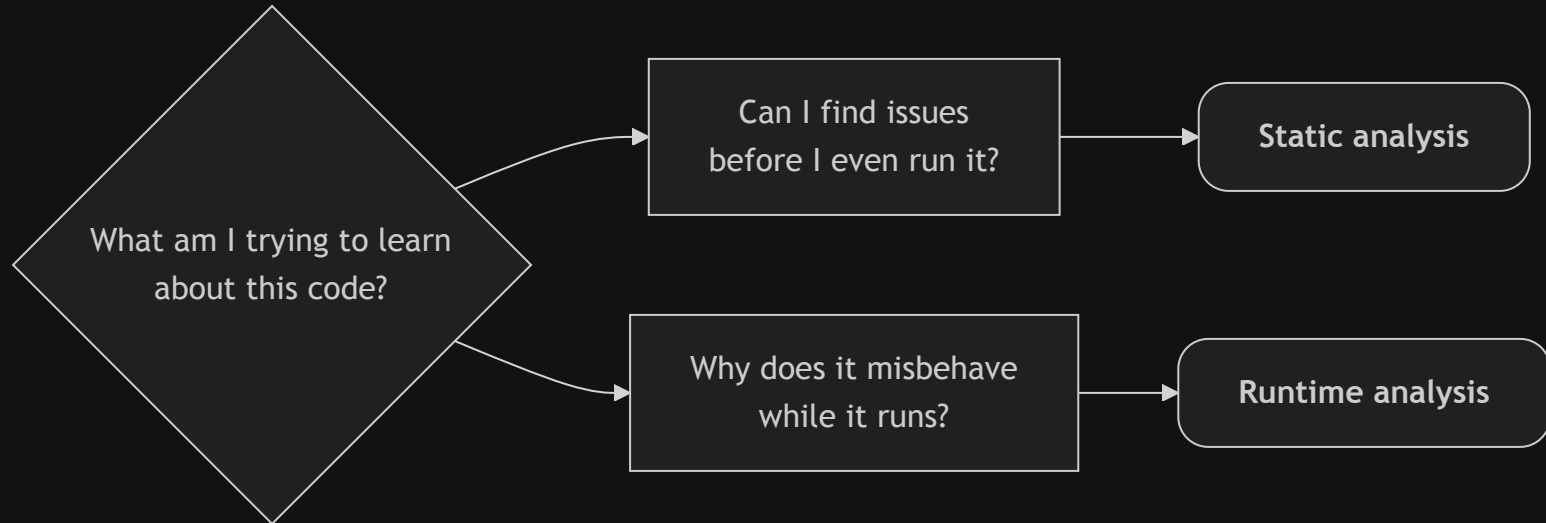
	ASan	LSan	TSan	UBSan
ASan	-	✓	✗	✓
LSan	✓	-	✗	✓
TSan	✗	✗	-	✓
UBSan	✓	✓	✓	-

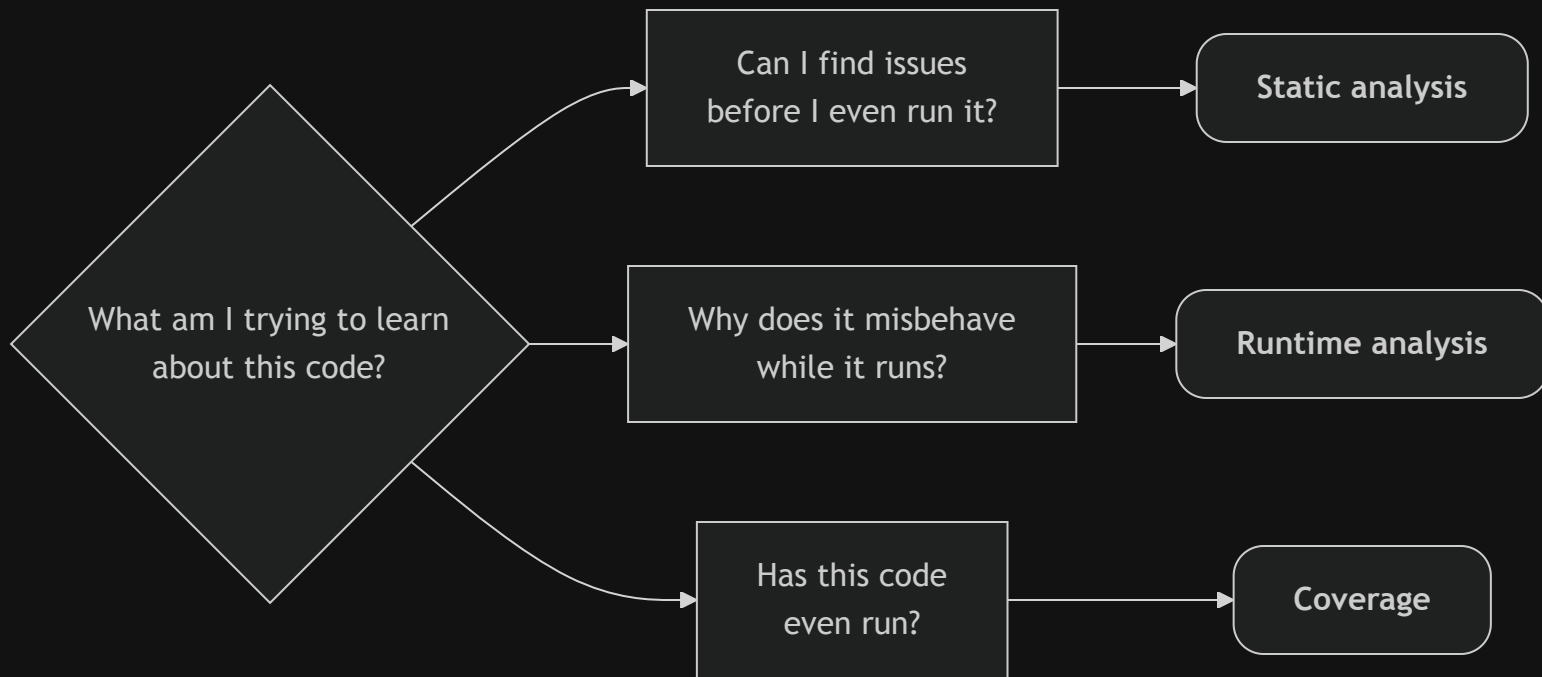
- UBSan stacks with everything
- Shadow-memory clashes (ASan, TSan)
- LSan is unavailable on Apple Silicon
- MSVC `/fsanitize` is ASan-only

Can I turn them all on at once?

	ASan	LSan	TSan	UBSan
ASan	-	✓	✗	✓
LSan	✓	-	✗	✓
TSan	✗	✗	-	✓
UBSan	✓	✓	✓	-

- UBSan stacks with everything
- Shadow-memory clashes (ASan, TSan)
- LSan is unavailable on Apple Silicon
- MSVC `/fsanitize` is ASan-only
- Typical CI split: one ASan+LSan+UBSan job, one TSan job





Coverage

Using coverage checks, you get to:

Coverage

Using coverage checks, you get to:

- Understand which parts of your code have been executed

Coverage

Using coverage checks, you get to:

- Understand which parts of your code have been executed
- Identify untested code paths

Coverage

Using coverage checks, you get to:

- Understand which parts of your code have been executed
- Identify untested code paths
- Get a cool badge for your repo's README:

A badge with a dark gray background and a green bar on the right. The word "coverage" is in white text on the gray background, and "97%" is in white text on the green bar.

coverage 97%

Source-based coverage

```
clang++ -std=c++20 -fprofile-instr-generate -fcoverage-mapping stats.cpp main.cpp -o humidity-stats
LLVM_PROFILE_FILE=readings.profraw ./humidity-stats readings.txt
llvm-profdata merge -sparse *.profraw -o merged.profdata
llvm-cov report ./humidity-stats -instr-profile merged.profdata
```

File	Functions	Lines	Branches
stats.cpp	100.00 %	100.00 %	100.00 %
main.cpp	100.00 %	93.48 %	85.71 %

Source-based coverage

```
clang++ -std=c++20 -fprofile-instr-generate -fcoverage-mapping stats.cpp main.cpp -o humidity-stats
LLVM_PROFILE_FILE=readings.profraw ./humidity-stats readings.txt
llvm-profdata merge -sparse *.profraw -o merged.profdata
llvm-cov report ./humidity-stats -instr-profile merged.profdata
```

File	Functions	Lines	Branches
stats.cpp	100.00 %	100.00 %	100.00 %
main.cpp	100.00 %	93.48 %	85.71 %

- main.cpp : 3 lines and 2 branch outcomes that *nothing* has ever executed

Where coverage earns its keep

```
llvm-cov show ./humidity-stats -instr-profile merged.profdata main.cpp
```


Where coverage earns its keep

```
llvm-cov show ./humidity-stats -instr-profile merged.profdata main.cpp
```

```
53|      3|      if (readings.empty())
54|      0|      {
55|      0|          std::cout << "warning: no readings loaded\n";
56|      0|      }
57|      |
58|      3|      printReport(computeStats(readings));
```

Where coverage earns its keep

```
llvm-cov show ./humidity-stats -instr-profile merged.profdata main.cpp
```

```
53|      3|      if (readings.empty())
54|      0|      {
55|      0|          std::cout << "warning: no readings loaded\n";
56|      0|      }
57|      |
58|      3|      printReport(computeStats(readings));
```

- Execution count 0 : We forgot to test the empty file case

Where coverage earns its keep

```
llvm-cov show ./humidity-stats -instr-profile merged.profdata main.cpp
```

```
53|      3|      if (readings.empty())
54|      0|      {
55|      0|          std::cout << "warning: no readings loaded\n";
56|      0|      }
57|      |
58|      3|      printReport(computeStats(readings));
```

- Execution count `0` : We forgot to test the empty file case
- The other never-taken outcome is `r < 0` in the histogram

Running what never ran

One binary carrying coverage and UBSan

```
./ss-ubsan-cov empty.txt
```

Running what never ran

One binary carrying coverage and UBSan

```
./ss-ubsan-cov empty.txt
```

```
stats.cpp:21:20: runtime error: division by zero  
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior stats.cpp:21:20  
warning: no readings loaded  
min 2147483647 max -2147483648 mean 0
```

Running what never ran

One binary carrying coverage and UBSan

```
./ss-ubsan-cov empty.txt
```

```
stats.cpp:21:20: runtime error: division by zero  
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior stats.cpp:21:20  
warning: no readings loaded  
min 2147483647 max -2147483648 mean 0
```

- The branch now executed, but forgot to `return`

Running what never ran

One binary carrying coverage and UBSan

```
./ss-ubsan-cov empty.txt
```

```
stats.cpp:21:20: runtime error: division by zero
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior stats.cpp:21:20
warning: no readings loaded
min 2147483647 max -2147483648 mean 0
```

- The branch now executed, but forgot to `return`
- `sum / count` with `count = 0`

Running what never ran

One binary carrying coverage and UBSan

```
./ss-ubsan-cov empty.txt
```

```
stats.cpp:21:20: runtime error: division by zero
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior stats.cpp:21:20
warning: no readings loaded
min 2147483647 max -2147483648 mean 0
```

- The branch now executed, but forgot to `return`
- `sum / count` with `count = 0`
 - arm64 returned 0

The fix coverage was pointing at

C++ main.cpp

```
1  if (readings.empty())
2  {
3      std::cout << "warning: no readings loaded\n";
4  }
5
6  printReport(computeStats(readings));
```

The fix coverage was pointing at

C++ main.cpp

```
1  if (readings.empty())
2  {
3      std::cout << "warning: no readings loaded\n";
4      return 0;
5  }
6
7  printReport(computeStats(readings));
```

The fix coverage was pointing at

C++ main.cpp

```
1 if (readings.empty())
2 {
3     std::cout << "warning: no readings loaded\n";
4     return 0;
5 }
6
7 printReport(computeStats(readings));
```

```
./ss-fixed empty.txt
```

```
warning: no readings loaded
```

exit code `0`, UBSan silent

Coverage is a map, not a grade

100% covered $\neq$ 100% correct

Coverage is a map, not a grade

100% covered $\neq$ 100% correct

- The division ran in every single run, just never with `count = 0`

Coverage is a map, not a grade

100% covered $\neq$ 100% correct

- The division ran in every single run, just never with `count = 0`
- Easy to satisfy without actually testing anything meaningful

Coverage is a map, not a grade

100% covered $\neq$ 100% correct

- The division ran in every single run, just never with `count = 0`
- Easy to satisfy without actually testing anything meaningful
- Branch coverage is stricter than line coverage

Coverage is a map, not a grade

100% covered $\neq$ 100% correct

- The division ran in every single run, just never with `count == 0`
- Easy to satisfy without actually testing anything meaningful
- Branch coverage is stricter than line coverage
 - `if (x > 0) { return 1; }` is one line either way

Checks in Production

Assumptions the program can enforce itself

Where do assumptions live today?

- "readings are between 0 and 100": a comment at best

Where do assumptions live today?

- "readings are between 0 and 100": a comment at best
- "the vector must not be empty": someone's head

Where do assumptions live today?

- "readings are between 0 and 100": a comment at best
- "the vector must not be empty": someone's head
- "returns an owning pointer": tribal knowledge

Where do assumptions live today?

- "readings are between 0 and 100": a comment at best
- "the vector must not be empty": someone's head
- "returns an owning pointer": tribal knowledge

These can't be type-checked, linted, or enforced: **tools can't see intent**

C++26 contracts

C++ stats.cpp

```
1 int meanOf(const std::vector<int>& readings)
2 {
3     assert(!readings.empty());
4     int sum = 0;
5     for (int r : readings) sum += r;
6     return sum / readings.size();
7 }
```

C++26 contracts

C++ stats.cpp

```
1 int meanOf(const std::vector<int>& readings)
2     pre (!readings.empty())
3 {
4     int sum = 0;
5     for (int r : readings) sum += r;
6     return sum / readings.size();
7 }
```

C++26 contracts

C++ stats.cpp

```
1 int meanOf(const std::vector<int>& readings)
2     pre (!readings.empty())
3 {
4     int sum = 0;
5     for (int r : readings) sum += r;
6     return sum / readings.size();
7 }
```

- The requirement is part of the declaration

C++26 contracts

C++ stats.cpp

```
1 int meanOf(const std::vector<int>& readings)
2     pre (!readings.empty())
3 {
4     int sum = 0;
5     for (int r : readings) sum += r;
6     return sum / readings.size();
7 }
```

- The requirement is part of the declaration
 - Callers see it, tools see it, and the check has selectable semantics

Evaluation semantics

```
g++ -std=c++26 -fcontract-evaluation-semantic=< ... > stats.cpp
```

Evaluation semantics

```
g++ -std=c++26 -fcontract-evaluation-semantic=< ... > stats.cpp
```

Semantic	On violation	<code>meanOf</code> on empty input
<code>ignore</code>	no check emitted	UB

Evaluation semantics

```
g++ -std=c++26 -fcontract-evaluation-semantic=< ... > stats.cpp
```

Semantic	On violation	<code>meanOf</code> on empty input
<code>ignore</code>	no check emitted	UB
<code>observe</code>	log, continue	logs, then UB

Evaluation semantics

```
g++ -std=c++26 -fcontract-evaluation-semantic=< ... > stats.cpp
```

Semantic	On violation	<code>meanOf</code> on empty input
<code>ignore</code>	no check emitted	UB
<code>observe</code>	log, continue	logs, then UB
<code>enforce</code>	log, terminate	stopped

Evaluation semantics

```
g++ -std=c++26 -fcontract-evaluation-semantic=< ... > stats.cpp
```

Semantic	On violation	<code>meanOf</code> on empty input
<code>ignore</code>	no check emitted	UB
<code>observe</code>	log, continue	logs, then UB
<code>enforce</code>	log, terminate	stopped
<code>quick_enforce</code>	terminate	stopped

Caller checks the precondition

C++ stats.cpp

```
1 int main()
2 {
3     std::vector<int> readings;
4     std::cout << "mean " << meanOf(readings) << '\n';
5 }
```

Caller checks the precondition

C++ stats.cpp

```
1 int main()
2 {
3     std::vector<int> readings;
4     std::cout << "mean " << meanOf(readings) << '\n';
5 }
```

```
contract violation in function int meanOf(...) at example.cpp:5: !readings.empty()
[assertion_kind: pre, semantic: enforce, mode: predicate_false, terminating: yes]
```


Caller checks the precondition

C++ stats.cpp

```
1  int main()
2  {
3      std::vector<int> readings;
4      if (readings.empty())
5      {
6          std::cout << "no readings\n";
7          return 0;
8      }
9      std::cout << "mean " << meanOf(readings) << '\n';
10 }
```

"Hardening allows turning some instances of undefined behavior in the standard library into a contract violation."

- P3471R4, *Standard Library Hardening*

The hardened standard library

C++26 (P3471, Standard Library Hardening)

The hardened standard library

C++26 (P3471, Standard Library Hardening)

- `operator[]` requires `n < size()`

The hardened standard library

C++26 (P3471, Standard Library Hardening)

- `operator[]` requires `n < size()`
- Library preconditions become hardened preconditions

The hardened standard library

C++26 (P3471, Standard Library Hardening)

- `operator[]` requires `n < size()`
- Library preconditions become hardened preconditions
 - checked like a `pre`

The hardened standard library

C++26 (P3471, Standard Library Hardening)

- `operator[]` requires `n < size()`
- Library preconditions become hardened preconditions
 - checked like a `pre`

Modes of hardening:

Mode	Checks	Intended use
FAST	valid element access, valid input range	production
EXTENSIVE	+ null pointers, deallocation, ...	production, more budget
DEBUG	+ semantic requirements, verbose message	development

Boundary checks in production

If our program had shipped with hardening:

```
.../array:277: libc++ Hardening assertion __n < _Size failed:  
out-of-bounds access in std::array<T, N>
```


Boundary checks in production

If our program had shipped with hardening:

```
.../array:277: libc++ Hardening assertion __n < _Size failed:  
out-of-bounds access in std::array<T, N>
```

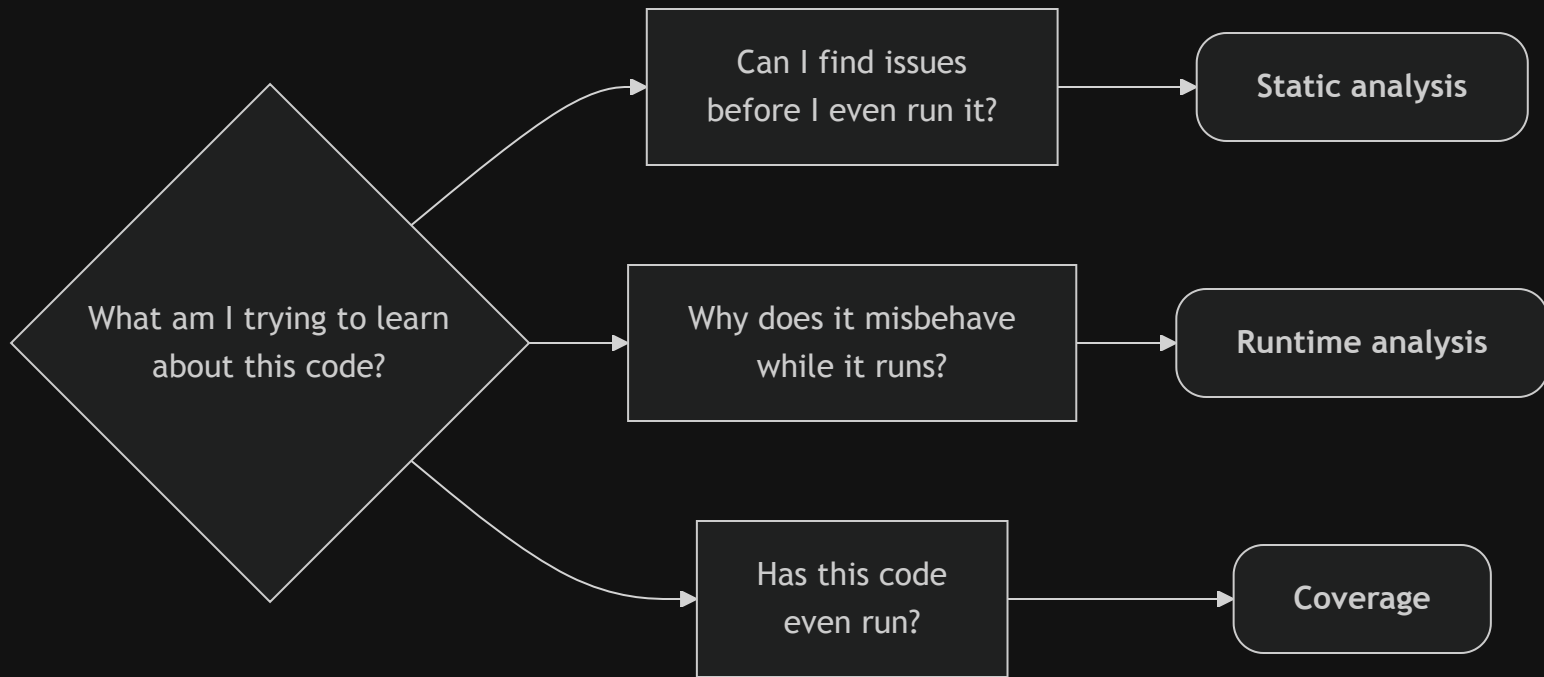
- `__n < _Size`: the `operator[]` precondition, enforced

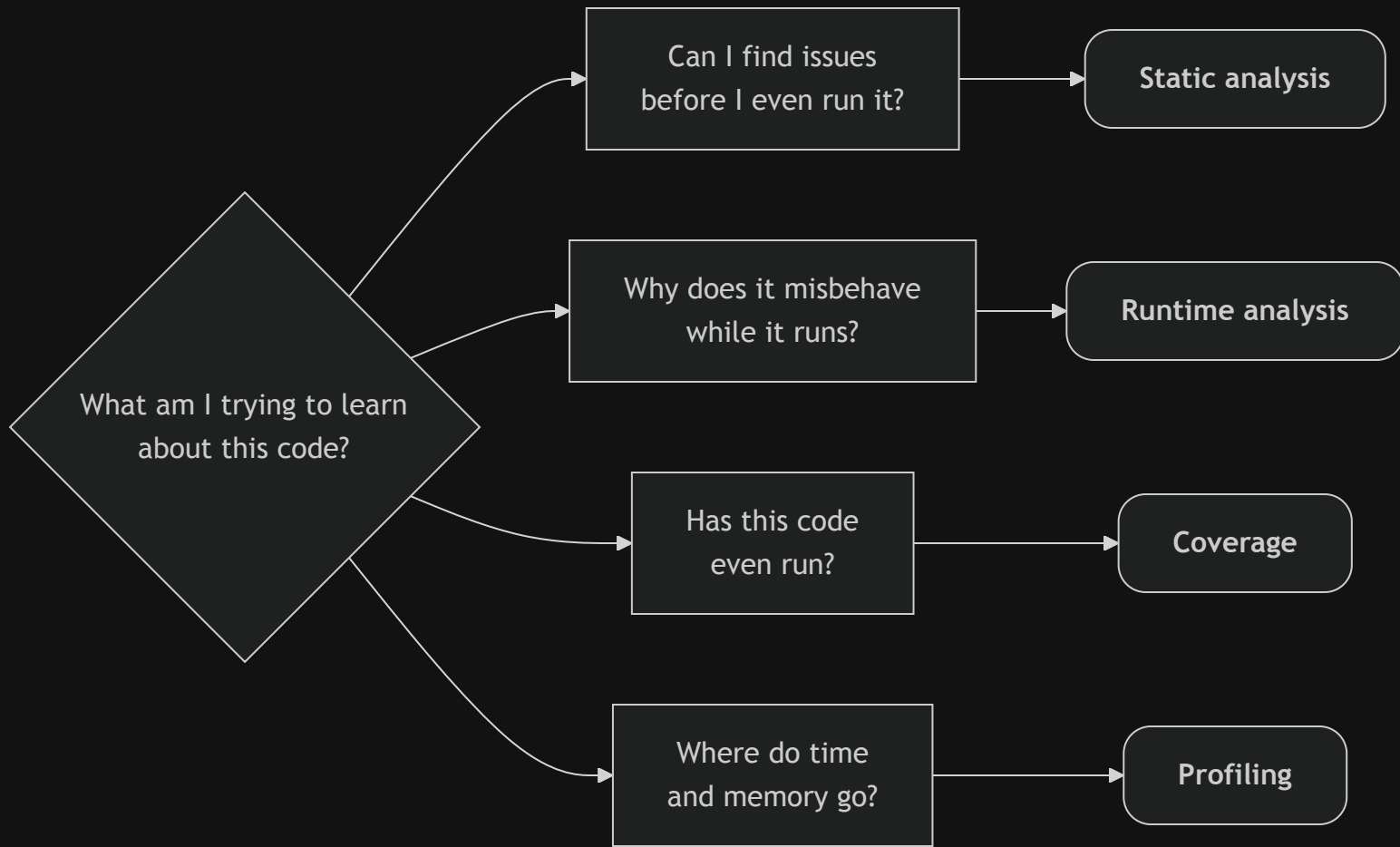
Boundary checks in production

If our program had shipped with hardening:

```
.../array:277: libc++ Hardening assertion __n < _Size failed:  
out-of-bounds access in std::array<T, N>
```

- `__n < _Size`: the `operator[]` precondition, enforced
- Google reports overhead "as low as 0.3 %" in deployment and 1000+ bugs found (P3471R4 §4)





Correct $\neq$ fast

```
time -p ./humidity-stats big.txt      # 20,000,000 readings, 55 MB
```

Correct $\neq$ fast

```
time -p ./humidity-stats big.txt      # 20,000,000 readings, 55 MB
```

```
min 0 max 100 mean 50
```

```
distinct 101
```

```
...
```

```
real 2.02
```

```
user 2.01
```

```
sys 0.01
```

Correct $\neq$ fast

```
time -p ./humidity-stats big.txt      # 20,000,000 readings, 55 MB
```

```
min 0 max 100 mean 50
```

```
distinct 101
```

```
...
```

```
real 2.02
```

```
user 2.01
```

```
sys 0.01
```

- Parsing 55 MB?

Correct $\neq$ fast

```
time -p ./humidity-stats big.txt      # 20,000,000 readings, 55 MB
```

```
min 0 max 100 mean 50
```

```
distinct 101
```

```
...
```

```
real 2.02
```

```
user 2.01
```

```
sys 0.01
```

- Parsing 55 MB?
- The histogram?

Correct $\neq$ fast

```
time -p ./humidity-stats big.txt      # 20,000,000 readings, 55 MB
```

```
min 0 max 100 mean 50
```

```
distinct 101
```

```
...
```

```
real 2.02
```

```
user 2.01
```

```
sys 0.01
```

- Parsing 55 MB?
- The histogram?
- The stats?

Correct $\neq$ fast

```
time -p ./humidity-stats big.txt      # 20,000,000 readings, 55 MB
```

```
min 0 max 100 mean 50
```

```
distinct 101
```

```
...
```

```
real 2.02
```

```
user 2.01
```

```
sys 0.01
```

- Parsing 55 MB?
- The histogram?
- The stats?

Don't guess.

"It is often a mistake to make a priori judgments about what parts of a program are really critical, since the universal experience of programmers who have been using measurement tools has been that their intuitive guesses fail."

- Donald E. Knuth, *Structured Programming with go to Statements* (1974)

Two ways to profile

Sampling

Instrumentation

Two ways to profile

Sampling

- Interrupt N times, record the call stack

Instrumentation

- Insert counting code around every call / allocation

Two ways to profile

Sampling

- Interrupt N times, record the call stack
- Low overhead

Instrumentation

- Insert counting code around every call / allocation
- High overhead (2-50×), can perturb timing

Two ways to profile

Sampling

- Interrupt N times, record the call stack
- Low overhead
- Statistical: percentages, not counts

Instrumentation

- Insert counting code around every call / allocation
- High overhead (2-50×), can perturb timing
- **Exact** counts

Two ways to profile

Sampling

- Interrupt N times, record the call stack
- Low overhead
- Statistical: percentages, not counts
- No rebuild needed

Instrumentation

- Insert counting code around every call / allocation
- High overhead (2-50×), can perturb timing
- **Exact** counts
- Rebuild or binary-translation runtime

Two ways to profile

Sampling

- Interrupt N times, record the call stack
- Low overhead
- Statistical: percentages, not counts
- No rebuild needed
- *perf* (Linux), macOS `sample`

Instrumentation

- Insert counting code around every call / allocation
- High overhead (2-50×), can perturb timing
- **Exact** counts
- Rebuild or binary-translation runtime
- *callgrind*, *gprof*, *heaptrack*

CPU profiling with `perf`

```
perf record [ ... ] ./ss-slow-linux big.txt
```

CPU profiling with `perf`

```
perf record [ ... ] ./ss-slow-linux big.txt
```

```
[ perf record: Captured and wrote 87.1 MB perf.data (10687 samples) ]
```

CPU profiling with `perf`

```
perf record [ ... ] ./ss-slow-linux big.txt
```

```
[ perf record: Captured and wrote 87.1 MB perf.data (10687 samples) ]
```

```
perf report --no-children
```

CPU profiling with `perf`

```
perf record [ ... ] ./ss-slow-linux big.txt
```

```
[ perf record: Captured and wrote 87.1 MB perf.data (10687 samples) ]
```

```
perf report --no-children
```

```
65.67%  countDistinct(const std::vector<int>&)  
12.57%  std::num_get<>::_M_extract_int<long>  
 5.18%  std::istream::sentry::sentry  
 2.69%  std::istream::operator>>(int&)  
 1.08%  histogram(const std::vector<int>&)  
 0.93%  computeStats(const std::vector<int>&)
```

CPU profiling with `perf`

```
perf record [ ... ] ./ss-slow-linux big.txt
```

```
[ perf record: Captured and wrote 87.1 MB perf.data (10687 samples) ]
```

```
perf report --no-children
```

```
65.67%  countDistinct(const std::vector<int>&)  
12.57%  std::num_get<>::_M_extract_int<long>  
 5.18%  std::istream::sentry::sentry  
 2.69%  std::istream::operator>>(int&)  
 1.08%  histogram(const std::vector<int>&)  
 0.93%  computeStats(const std::vector<int>&)
```

The culprit: `countDistinct`

C++ stats.cpp

```
1  int countDistinct(const std::vector<int>& readings)
2  {
3      int distinct = 0;
4      for (std::size_t i = 0; i < readings.size(); ++i)
5      {
6          bool seen = false;
7          for (std::size_t j = 0; j < i; ++j)
8          {
9              if (readings[j] == readings[i]) { seen = true; break; }
10             }
11             if (!seen) ++distinct;
12         }
13         return distinct;
14     }
```

The culprit: `countDistinct`

C++ stats.cpp

```
1  int countDistinct(const std::vector<int>& readings)
2  {
3      int distinct = 0;
4      for (std::size_t i = 0; i < readings.size(); ++i)
5      {
6          bool seen = false;
7          for (std::size_t j = 0; j < i; ++j)
8          {
9              if (readings[j] == readings[i]) { seen = true; break; }
10             }
11             if (!seen) ++distinct;
12         }
13         return distinct;
14     }
```

- $O(n \cdot d)$ ($d = 101$ distinct values)

The culprit: countDistinct

C++ stats.cpp

```
1  int countDistinct(const std::vector<int>& readings)
2  {
3      int distinct = 0;
4      for (std::size_t i = 0; i < readings.size(); ++i)
5      {
6          bool seen = false;
7          for (std::size_t j = 0; j < i; ++j)
8          {
9              if (readings[j] == readings[i]) { seen = true; break; }
10             }
11             if (!seen) ++distinct;
12         }
13         return distinct;
14     }
```

- $O(n \cdot d)$ ($d = 101$ distinct values)
- 20 M readings $\times$ ~ 101 distinct values $\approx 2 \cdot 10^9$ comparisons

The fix: one hash-set pass

C++ stats.cpp

```
1  int countDistinct(const std::vector<int>& readings)
2  {
3      int distinct = 0;
4      for (std::size_t i = 0; i < readings.size(); ++i)
5      {
6          bool seen = false;
7          for (std::size_t j = 0; j < i; ++j)
8          {
9              if (readings[j] == readings[i]) { seen = true; break; }
10         }
11         if (!seen) ++distinct;
12     }
13     return distinct;
14 }
```

The fix: one hash-set pass

C++ stats.cpp

```
1 int countDistinct(const std::vector<int>& readings)
2 {
3     const std::unordered_set<int> seen(readings.begin(), readings.end());
4     return seen.size();
5 }
```

Measure again

	before	after	
<code>countDistinct</code>	1335 ms	24 ms	
parse / load	669 ms	660 ms	
total (wall)	2.02 s	0.71 s	~2.8× faster

Measure again

	before	after	
<code>countDistinct</code>	1335 ms	24 ms	
parse / load	669 ms	660 ms	
total (wall)	2.02 s	0.71 s	~2.8× faster

- The next hot spot is parsing (93 % of what is left)

Measure again

	before	after	
<code>countDistinct</code>	1335 ms	24 ms	
parse / load	669 ms	660 ms	
total (wall)	2.02 s	0.71 s	~2.8× faster

- The next hot spot is parsing (93 % of what is left)
- Measure → change → re-measure

Where does the *memory* go?

Where does the *memory* go?

- A memory profiler answers "where are the bytes"

Where does the *memory* go?

- A memory profiler answers "where are the bytes"
- `heaptrack` intercepts every `malloc` / `free` (instrumentation)
 - peak heap
 - allocation counts
 - leaks

Where does the *memory* go?

- A memory profiler answers "where are the bytes"
- `heaptrack` intercepts every `malloc` / `free` (instrumentation)
 - peak heap
 - allocation counts
 - leaks

```
heaptrack ./ss-fast-linux big.txt  
heaptrack_print heaptrack.ss-fast-linux.*.gz
```

The fix wasn't free

	slow build	fast build	
allocations	31	136	← 4.4x
peak heap	201.41 MB	201.41 MB	
peak RSS	131 MiB	133 MiB	

The fix wasn't free

	slow build	fast build	
allocations	31	136	← 4.4x
peak heap	201.41 MB	201.41 MB	
peak RSS	131 MiB	133 MiB	

- Peak memory is dominated by the data itself

The fix wasn't free

	slow build	fast build	
allocations	31	136	← 4.4x
peak heap	201.41 MB	201.41 MB	
peak RSS	131 MiB	133 MiB	

- Peak memory is dominated by the data itself
- The cost moved into allocation count

Real Projects

Your codebase already exists

- Every tool you enable can find hundreds of problems on day one

```
clang-tidy main.cpp --checks='*' -- -std=c++20
```

Your codebase already exists

- Every tool you enable can find hundreds of problems on day one

```
clang-tidy main.cpp --checks='*' -- -std=c++20
```

```
main.cpp:3:1: warning: system include algorithm not allowed [llvmlibc-restrict-system-libc-headers]
main.cpp:10:18: warning: use a trailing return type for this function [modernize-use-trailing-return-type]
main.cpp:10:18: warning: function 'loadReadings' can be made static or moved into an anonymous namespace to enforce inte
...
11324 warnings generated.
```


Your codebase already exists

- Every tool you enable can find hundreds of problems on day one

```
clang-tidy main.cpp --checks='*' -- -std=c++20
```

```
main.cpp:3:1: warning: system include algorithm not allowed [llvmlibc-restrict-system-libc-headers]
main.cpp:10:18: warning: use a trailing return type for this function [modernize-use-trailing-return-type]
main.cpp:10:18: warning: function 'loadReadings' can be made static or moved into an anonymous namespace to enforce inte
...
11324 warnings generated.
```

- **65 warnings from 17 checks** on our 65-line `main.cpp`

Your codebase already exists

- Every tool you enable can find hundreds of problems on day one

```
clang-tidy main.cpp --checks='*' -- -std=c++20
```

```
main.cpp:3:1: warning: system include algorithm not allowed [llvmlibc-restrict-system-libc-headers]
main.cpp:10:18: warning: use a trailing return type for this function [modernize-use-trailing-return-type]
main.cpp:10:18: warning: function 'loadReadings' can be made static or moved into an anonymous namespace to enforce inte
...
11324 warnings generated.
```

- **65 warnings from 17 checks** on our 65-line `main.cpp`
- The goal is no new findings

The adoption staircase

The adoption staircase

1. Compile clean at `-Wall -Wextra`

The adoption staircase

1. Compile clean at `-Wall -Wextra`
2. `-Werror`

The adoption staircase

1. Compile clean at `-Wall -Wextra`
2. `-Werror`
3. Useful clang-tidy check sets

The adoption staircase

1. Compile clean at `-Wall -Wextra`
2. `-Werror`
3. Useful clang-tidy check sets
4. Sanitizers on the test suite

The adoption staircase

1. Compile clean at `-Wall -Wextra`
2. `-Werror`
3. Useful clang-tidy check sets
4. Sanitizers on the test suite
5. Observed coverage and profiling

The adoption staircase

1. Compile clean at `-Wall -Wextra`
2. `-Werror`
3. Useful clang-tidy check sets
4. Sanitizers on the test suite
5. Observed coverage and profiling

Suppressing: narrow, named, explained

C++ widget.cpp

```
1 int n = readings.size();
```

Suppressing: narrow, named, explained

C++ widget.cpp

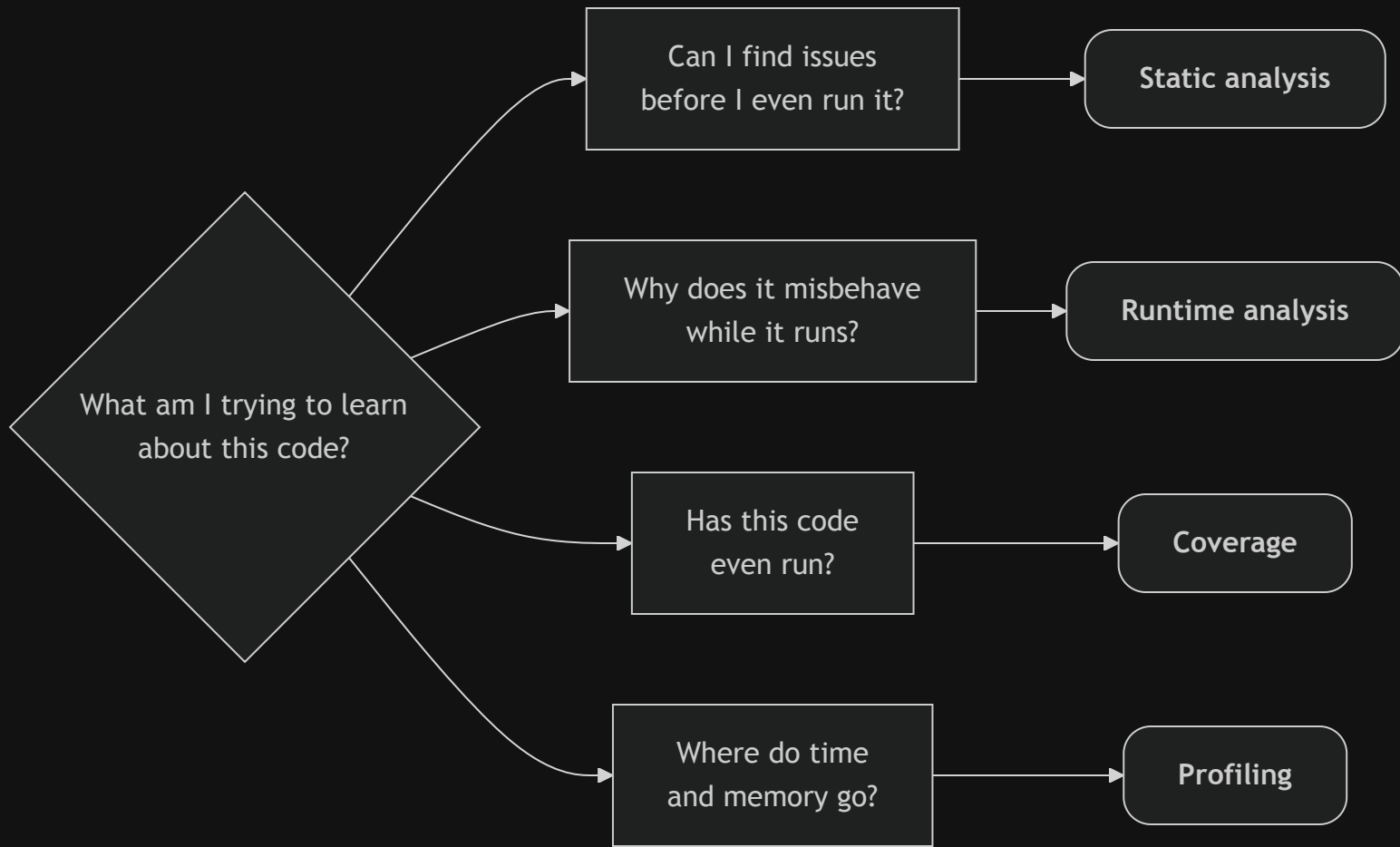
```
1 int n = readings.size();
```

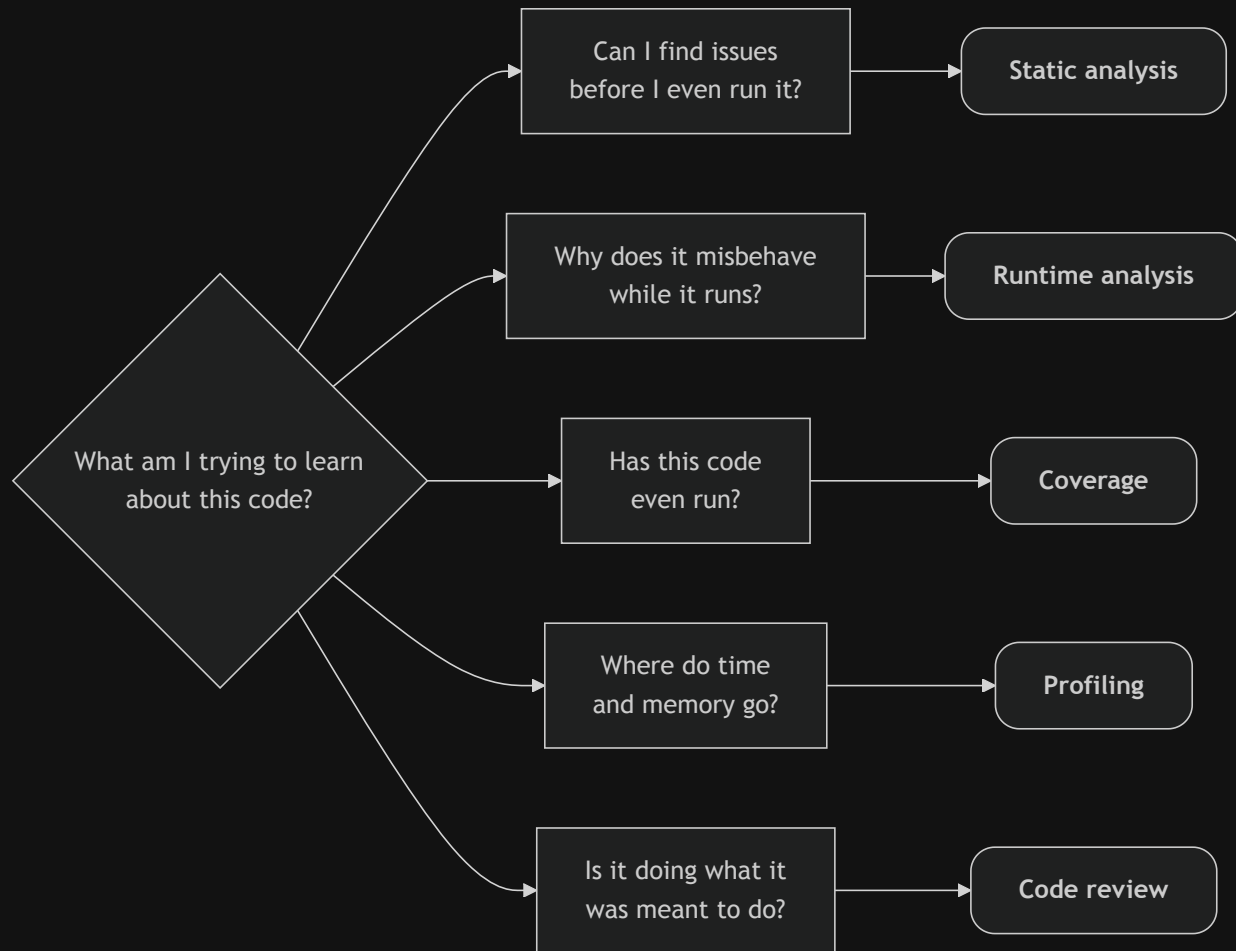
warning: narrowing conversion from 'size_type' (aka
'unsigned long') to signed type 'int' [bugprone-narrowing-conversions]

Suppressing: narrow, named, explained

C++ widget.cpp

```
1 // NOLINTNEXTLINE(bugprone-narrowing-conversions): legacy code, not touched by the new policy
2 int n = readings.size();
```





Is this code doing what it was meant to do?

Back to the *original* listing, before any tool touched it:

C++ main.cpp

```
if (readings.empty())
{
    std::cout << "warning: no readings loaded\n";
}

printReport(computeStats(readings), false);
```

Is this code doing what it was meant to do?

Back to the *original* listing, before any tool touched it:

C++ main.cpp

```
if (readings.empty())  
{  
    std::cout << "warning: no readings loaded\n";  
}  
  
printReport(computeStats(readings), false);
```

- Warn and then compute the stats anyway?

Is this code doing what it was meant to do?

Back to the *original* listing, before any tool touched it:

C++ main.cpp

```
if (readings.empty())
{
    std::cout << "warning: no readings loaded\n";
}

printReport(computeStats(readings), false);
```

- Warn and then compute the stats anyway?
- What is that `false` ?

Is this code doing what it was meant to do?

Back to the *original* listing, before any tool touched it:

C++ main.cpp

```
if (readings.empty())
{
    std::cout << "warning: no readings loaded\n";
}

printReport(computeStats(readings), false);
```

- Warn and then compute the stats anyway?
- What is that `false` ?

But reviewers are inconsistent...

Is this code doing what it was meant to do?

Back to the *original* listing, before any tool touched it:

C++ main.cpp

```
if (readings.empty())
{
    std::cout << "warning: no readings loaded\n";
}

printReport(computeStats(readings), false);
```

- Warn and then compute the stats anyway?
- What is that `false` ?

But reviewers are inconsistent...

- Tired by the tenth file

Is this code doing what it was meant to do?

Back to the *original* listing, before any tool touched it:

C++ main.cpp

```
if (readings.empty())
{
    std::cout << "warning: no readings loaded\n";
}

printReport(computeStats(readings), false);
```

- Warn and then compute the stats anyway?
- What is that `false` ?

But reviewers are inconsistent...

- Tired by the tenth file
- Lazier on a Friday

Is this code doing what it was meant to do?

Back to the *original* listing, before any tool touched it:

C++ main.cpp

```
if (readings.empty())
{
    std::cout << "warning: no readings loaded\n";
}

printReport(computeStats(readings), false);
```

- Warn and then compute the stats anyway?
- What is that `false` ?

But reviewers are inconsistent...

- Tired by the tenth file
- Lazier on a Friday
- Blind to a 64 → 32 bit narrowing

What only a reader can enforce

There are many conventions no linter **should** formalize:

What only a reader can enforce

There are many conventions no linter **should** formalize:

- Layering rules, "warnings go to `stderr`", meaningful names

What only a reader can enforce

There are many conventions no linter **should** formalize:

- Layering rules, "warnings go to `stderr`", meaningful names
- Ownership semantics: who is responsible for deleting?

What only a reader can enforce

There are many conventions no linter **should** formalize:

- Layering rules, "warnings go to `stderr`", meaningful names
- Ownership semantics: who is responsible for deleting?
- Readable diagnostics

What only a reader can enforce

There are many conventions no linter **should** formalize:

- Layering rules, "warnings go to `stderr`", meaningful names
- Ownership semantics: who is responsible for deleting?
- Readable diagnostics
- No deterministic tool measures whether a message helps at 3 a.m.

What only a reader can enforce

There are many conventions no linter **should** formalize:

- Layering rules, "warnings go to `stderr`", meaningful names
- Ownership semantics: who is responsible for deleting?
- Readable diagnostics
- No deterministic tool measures whether a message helps at 3 a.m.

But can this be automated?

Reviewing with AI

"Find every problem by reading alone."

Reviewing with AI

"Find every problem by reading alone."

1. Out-of-bounds write in histogram - UB (memory corruption)
2. Division by zero on empty input - UB
3. Signed overflow when summing - UB
4. Memory leak / raw owning pointer - leak + API design
5. File-open failure is silently treated as "no readings" - robustness / bug
6. Malformed input silently truncates the data - robustness
7. Truncating narrowing conversions in computeStats - bug (portability/correctness)
8. countDistinct is $O(n^2)$ - performance
9. Dead/unused code - style (and a smell)
10. Include hygiene - style
11. Minor style / idiom

Reviewing with AI

"Find every problem by reading alone."

1. Out-of-bounds write in histogram - UB (memory corruption)
2. Division by zero on empty input - UB
3. Signed overflow when summing - UB
4. Memory leak / raw owning pointer - leak + API design
5. File-open failure is silently treated as "no readings" - robustness / bug
6. Malformed input silently truncates the data - robustness
7. Truncating narrowing conversions in computeStats - bug (portability/correctness)
8. countDistinct is $O(n^2)$ - performance
9. Dead/unused code - style (and a smell)
10. Include hygiene - style
11. Minor style / idiom

74 seconds, 11,622 tokens (\$0.43)

Reviewing with AI

"Find every problem by reading alone."

1. Out-of-bounds write in histogram - UB (memory corruption)
2. Division by zero on empty input - UB
3. Signed overflow when summing - UB
4. Memory leak / raw owning pointer - leak + API design
5. File-open failure is silently treated as "no readings" - robustness / bug
6. Malformed input silently truncates the data - robustness
7. Truncating narrowing conversions in computeStats - bug (portability/correctness)
8. countDistinct is $O(n^2)$ - performance
9. Dead/unused code - style (and a smell)
10. Include hygiene - style
11. Minor style / idiom

74 seconds, 11,622 tokens (\$0.43), and about 1.3 L of water

Planted bug	Caught in this talk by	Blind AI
unused variable <code>checksum</code>	<code>-Wall</code>	✓
unused parameter <code>verbose</code>	<code>-Wextra</code>	✓
<code>size()</code> → <code>int</code> narrowing	<code>-Wconversion</code> / clang-tidy	✓
include hygiene	include-what-you-use	✓
out-of-bounds histogram write	ASan	✓
signed overflow in <code>sum</code>	UBSan	✓
division by zero on empty input	UBSan	✓
leaked <code>loadReadings</code> vector	LeakSanitizer	✓

Same prompt, four models

Model	Findings	Identified	Missed	False claims	Tokens	Time	Cost	¢ / LoC
Fable 5	11	10/10	0	0	11.6 k	74 s	≈ \$0.43	0.4

Same prompt, four models

Model	Findings	Identified	Missed	False claims	Tokens	Time	Cost	¢ / LoC
Fable 5	11	10/10	0	0	11.6 k	74 s	≈ \$0.43	0.4
Opus 5	17	10/10	0	0	15.1 k	120 s	≈ \$0.31	0.3

Same prompt, four models

Model	Findings	Identified	Missed	False claims	Tokens	Time	Cost	¢ / LoC
Fable 5	11	10/10	0	0	11.6 k	74 s	≈ \$0.43	0.4
Opus 5	17	10/10	0	0	15.1 k	120 s	≈ \$0.31	0.3
Sonnet 5	15	9/10	1	0	14.7 k	40 s	≈ \$0.07	0.06

Same prompt, four models

Model	Findings	Identified	Missed	False claims	Tokens	Time	Cost	¢ / LoC
Fable 5	11	10/10	0	0	11.6 k	74 s	≈ \$0.43	0.4
Opus 5	17	10/10	0	0	15.1 k	120 s	≈ \$0.31	0.3
Sonnet 5	15	9/10	1	0	14.7 k	40 s	≈ \$0.07	0.06
Haiku 4.5	10	8/10	2	0	14.4 k	44 s	≈ \$0.05	0.04

Same prompt, four models

Model	Findings	Identified	Missed	False claims	Tokens	Time	Cost	¢ / LoC
Fable 5	11	10/10	0	0	11.6 k	74 s	≈ \$0.43	0.4
Opus 5	17	10/10	0	0	15.1 k	120 s	≈ \$0.31	0.3
Sonnet 5	15	9/10	1	0	14.7 k	40 s	≈ \$0.07	0.06
Haiku 4.5	10	8/10	2	0	14.4 k	44 s	≈ \$0.05	0.04

- Sonnet 5 missed the include hygiene, Haiku 4.5 also missed the signed overflow

Findings no deterministic tool reported

- Readings {1, 2} report mean 1 (silent truncation)

Findings no deterministic tool reported

- Readings {1, 2} report mean 1 (silent truncation)
- Malformed input: 12 abc 34 stops parsing at abc and silently drops the rest of the file

Findings no deterministic tool reported

- Readings {1, 2} report mean 1 (silent truncation)
- Malformed input: 12 abc 34 stops parsing at abc and silently drops the rest of the file
- Warn but proceed: misleading UX

Findings no deterministic tool reported

- Readings {1, 2} report mean 1 (silent truncation)
- Malformed input: 12 abc 34 stops parsing at abc and silently drops the rest of the file
- Warn but proceed: misleading UX
- verbose flagged as a boolean-flag anti-pattern

Human review cost

Published number

Value

Source

Human review cost

Published number	Value	Source
Effective review speed	200-400 LoC per 60-90 min 70-90 % of defects	SmartBear-Cisco study (2,500 reviews)

Human review cost

Published number	Value	Source
Effective review speed	200-400 LoC per 60-90 min 70-90 % of defects	SmartBear-Cisco study (2,500 reviews)
Median US developer pay	\$133,080 / yr $\approx$ \$64 / h	US BLS, May 2024

Human review cost

Published number	Value	Source
Effective review speed	200-400 LoC per 60-90 min 70-90 % of defects	SmartBear-Cisco study (2,500 reviews)
Median US developer pay	\$133,080 / yr $\approx$ \$64 / h	US BLS, May 2024
Full hourly cost	$\approx$ \$90 / h (adding benefits)	BLS ECEC

Human review cost

Published number	Value	Source
Effective review speed	200-400 LoC per 60-90 min 70-90 % of defects	SmartBear-Cisco study (2,500 reviews)
Median US developer pay	\$133,080 / yr $\approx$ \$64 / h	US BLS, May 2024
Full hourly cost	$\approx$ \$90 / h (adding benefits)	BLS ECEC

- Our 118 LoC: **18-53 minutes $\approx$ \$20-80**

Human review cost

Published number	Value	Source
Effective review speed	200-400 LoC per 60-90 min 70-90 % of defects	SmartBear-Cisco study (2,500 reviews)
Median US developer pay	\$133,080 / yr $\approx$ \$64 / h	US BLS, May 2024
Full hourly cost	$\approx$ \$90 / h (adding benefits)	BLS ECEC

- Our 118 LoC: **18-53 minutes $\approx$ \$20-80**
 - The AI runs: **40-120 seconds $\approx$ \$0.05-\$0.43**

Human review cost

Published number	Value	Source
Effective review speed	200-400 LoC per 60-90 min 70-90 % of defects	SmartBear-Cisco study (2,500 reviews)
Median US developer pay	\$133,080 / yr $\approx$ \$64 / h	US BLS, May 2024
Full hourly cost	$\approx$ \$90 / h (adding benefits)	BLS ECEC

- Our 118 LoC: **18-53 minutes $\approx$ \$20-80**
 - The AI runs: **40-120 seconds $\approx$ \$0.05-\$0.43**
- But human review brings what no model can: design pushback, context, **ownership**

Human review cost

Published number	Value	Source
Effective review speed	200-400 LoC per 60-90 min 70-90 % of defects	SmartBear-Cisco study (2,500 reviews)
Median US developer pay	\$133,080 / yr $\approx$ \$64 / h	US BLS, May 2024
Full hourly cost	$\approx$ \$90 / h (adding benefits)	BLS ECEC

- Our 118 LoC: **18-53 minutes $\approx$ \$20-80** (+30-90 mL of water)
 - The AI runs: **40-120 seconds $\approx$ \$0.05-\$0.43**
- But human review brings what no model can: design pushback, context, **ownership**

Analysis, tests, and review

- Checks assert what the program should do, code reviews verify how it does it

Analysis, tests, and review

- Checks assert what the program should do, code reviews verify how it does it
- Review catches design and intent problems no other tool can see

Analysis, tests, and review

- Checks assert what the program should do, code reviews verify how it does it
- Review catches design and intent problems no other tool can see
 - Tools scale to every line of every build, reviewers can't (and shouldn't!)

Conclusion

What code analysis proves

Output	What that detects
No warnings	none of the enabled checks fired
Sanitizers silent	no bug on the paths you ran
100 % coverage	every line ran
Flat profile	no hotspot in this workload
PR approved	no issues found by the reviewer

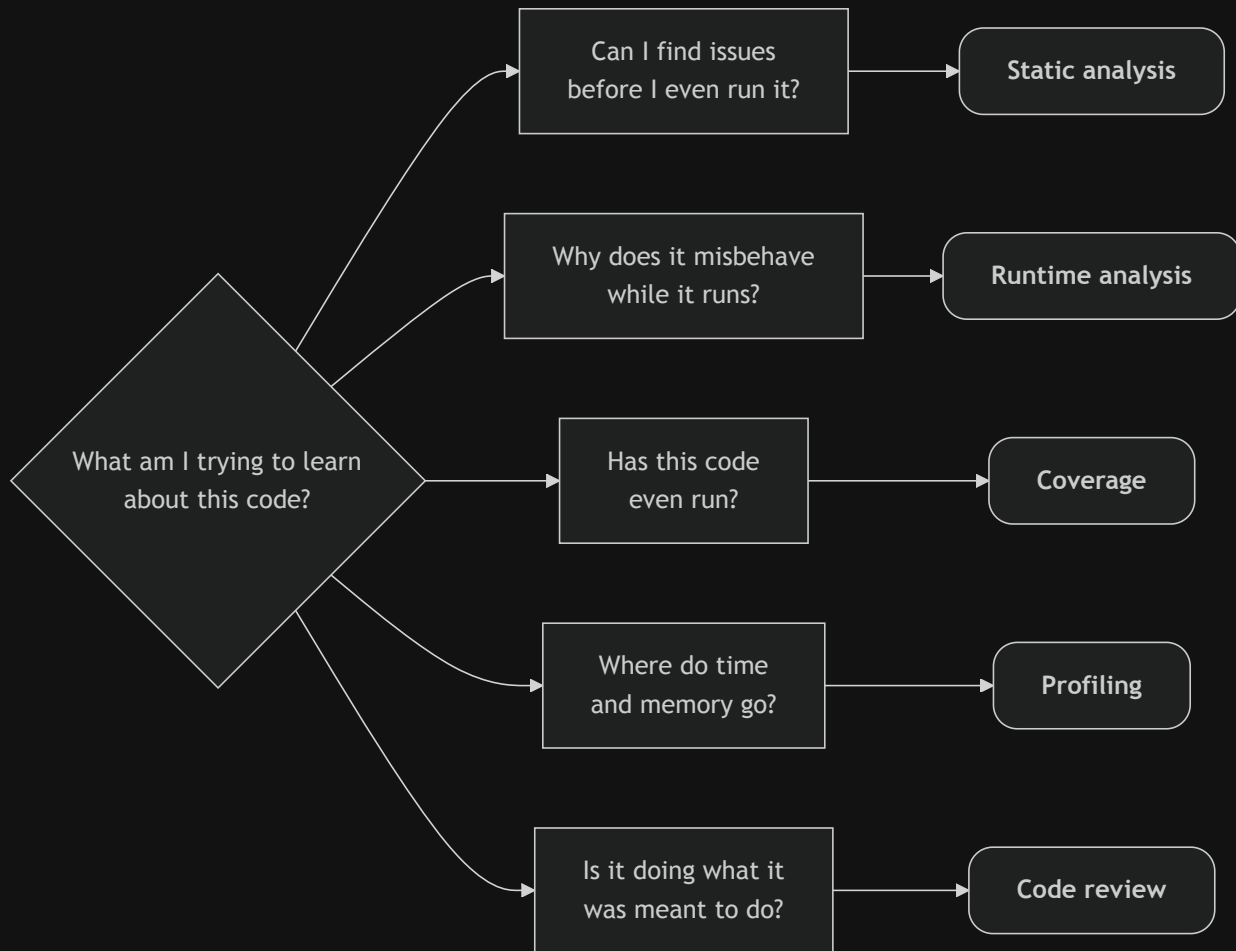
What code analysis proves

Output	What that detects	What it says nothing about
No warnings	none of the enabled checks fired	checks you did not enable
Sanitizers silent	no bug on the paths you ran	paths you did not run
100 % coverage	every line ran	whether any result was checked
Flat profile	no hotspot in this workload	what you did not measure
PR approved	no issues found by the reviewer	what the reviewer did not see

What code analysis proves

Output	What that detects	What it says nothing about
No warnings	none of the enabled checks fired	checks you did not enable
Sanitizers silent	no bug on the paths you ran	paths you did not run
100 % coverage	every line ran	whether any result was checked
Flat profile	no hotspot in this workload	what you did not measure
PR approved	no issues found by the reviewer	what the reviewer did not see

No single step replaces the others



Always has been

Wait, it's all
code analysis?



Thank you!